

PROYECTO NutriHuella
Arquitectura vista 4+1
Versión 1.0

Profesor: Fernando Herrera
Integrantes:
Gonzalo Troncoso
Leandro Valenzuela
Alexander Orell

22 de octubre 2025

Identificación de Documento

Identificación	Arquitectura vista 4+1
Proyecto	NutriHuella
Versión	1.0

Documento mantenido por	Gonzalo Troncoso
Fecha de última revisión	04/10/2025
Fecha de próxima revisión	22/10/2025

Documento aprobado por	Fernando Herrera
Fecha de última aprobación	22/10/2025

Historia de Revisiones

Fecha	Versión	Descripción	Autor
22/10/23	1.0	Entrega del documento.	Gonzalo Troncoso

Indice

1 Introducción.....	3
1.1 Contexto del Problema.....	4
1.2 Propósito.....	4
1.3 Ámbito.....	5
1.4 Definiciones, acrónimos y abreviaciones.....	6
1.5 Resumen ejecutivo.....	9
2 Metas y Restricciones de la Arquitectura.....	10
2.1 Metas de la arquitectura.....	10
2.2 Restricciones de la Arquitectura.....	12
2.3 Reglas de Negocio.....	13
A. Principios transversales.....	13
B. Usuarios y roles.....	13
C. Mascotas, especies y razas.....	13
D. Clínica (registro clínico, vacunas, enfermedades, pesos).....	13
E. Nutrición (perfil e histórico).....	14
F. Despensa (ingredientes).....	14
G. Recetas, favoritos y feedback.....	14
H. IA: generación y límites.....	14
I. Integridad, orden y concurrencia.....	15
J. Membresía y límites.....	15
3 Modelo 4+1.....	16
3.1 Vista lógica.....	16
Modelo E-R.....	16
Diagrama de Clase.....	18
Diagrama de Secuencia.....	20
3.2 Vista Despliegue.....	24
Diagrama de componentes de paquetes.....	24
3.3 Vista de Procesos.....	26
Diagrama de actividad.....	26
3.4 Vista de Fisica.....	32
Diagrama de despliegue.....	32
Descripción general.....	33
Componentes principales.....	33
Entorno y configuración.....	34
Resumen del flujo de despliegue.....	34
3.5 Vista de Escenarios/Caso de uso.....	35
Casos de uso.....	35
Escenarios (Funcionales).....	40
Reglas de negocio aplicables.....	47
Escenarios Calidad (No Funcionales).....	48
4 Patron.....	54
5 Conclusion.....	55

1 Introducción

NutriHuella es una plataforma web y móvil orientada a la **nutrición personalizada de mascotas** mediante **inteligencia artificial**. Integra ficha clínica, perfil nutricional, gestión de despensa y generación automática de **planes/recetas** ajustados a necesidades, objetivos y restricciones alimentarias, reduciendo la brecha entre recomendaciones genéricas y la personalización efectiva.

El sistema incorpora **autenticación** por correo y **Google OAuth** (tokens **JWT**), políticas de acceso y **cuotas diarias** según plan (**FREE/PREMIUM**). Además, contempla un flujo de **Membresía Premium** con integración a **pasarela de pago** para **upgrade** y **cancelación** programada al final del ciclo, asegurando continuidad del servicio y transparencia para el usuario.

La solución se apoya en principios de **escalabilidad, seguridad y mantenibilidad**, con separación clara entre interfaz, servicios de negocio, motor de IA y base de datos, junto con telemetría y manejo de errores para una experiencia confiable. Esta organización facilita la trazabilidad de decisiones y el control de calidad a lo largo del ciclo de vida.

Para estructurar la descripción y mantener coherencia entre requerimientos, diseño e implementación, se adopta el **modelo 4+1** de Kruchten —**lógica, de procesos, de desarrollo, física y de casos de uso**—, cuyas vistas se presentarán posteriormente sin profundizar aquí en su detalle. En conjunto, **NutriHuella** busca aportar valor tangible a tutores y profesionales, mejorando la adherencia nutricional y habilitando una evolución basada en evidencia y retroalimentación continua.

1.1 Contexto del Problema

En la actualidad, la alimentación de las mascotas se encuentra fuertemente influenciada por la **dependencia del pellet** como principal fuente nutricional. Este tipo de alimento, si bien es accesible y práctico, suele estar compuesto por productos **altamente procesados y de baja calidad nutricional**, lo que limita el bienestar y la salud a largo plazo de los animales de compañía.

A esta situación se suma una marcada **falta de conocimiento por parte de los dueños**, quienes en muchos casos desconocen las **necesidades nutricionales reales de sus mascotas**. La ausencia de orientación profesional o de herramientas accesibles que promuevan una alimentación balanceada dificulta la toma de decisiones informadas sobre su dieta.

Por otro lado, existe una **percepción generalizada de que la alimentación natural es costosa e inalcanzable**, lo que refuerza la preferencia por productos comerciales y reduce la adopción de alternativas más saludables. Esta percepción se ve acentuada por la falta de plataformas que permitan planificar, calcular y adaptar dietas naturales de manera personalizada y económica.

1.2 Propósito

El propósito de **NutriHuella** es **mejorar la calidad de vida de las mascotas** mediante una herramienta tecnológica que promueva una **alimentación sana, natural y accesible**, adaptada a las necesidades particulares de cada animal.

A través de una **plataforma web y móvil impulsada por inteligencia artificial**, NutriHuella busca facilitar la generación de **planes nutricionales personalizados**, basados en la **información clínica y preferencias alimenticias** de cada mascota. De esta manera, los dueños pueden acceder a recomendaciones precisas y confiables sin requerir conocimientos técnicos o veterinarios avanzados.

Además, la aplicación fomenta una **comunidad colaborativa** mediante un **recetario compartido**, donde los usuarios pueden **descubrir, crear y compartir recetas saludables**, fortaleciendo así la educación nutricional y el compromiso con el bienestar animal.

En conjunto, el proyecto pretende **reducir la dependencia del pellet, combatir la desinformación nutricional y derribar la percepción de que la alimentación natural es costosa**, contribuyendo a una tenencia más responsable, informada y sostenible.

1.3 Ámbito

Animal y la nutrición personalizada:

Buscando resolver las deficiencias actuales en la gestión alimenticia de mascotas, la falta de conocimiento nutricional y la ausencia de herramientas tecnológicas accesibles para los dueños. El software se orientará a los siguientes ámbitos funcionales:

Gestión de Usuarios:

Permitir el registro, autenticación y administración de usuarios mediante un sistema seguro con autenticación por correo o proveedor externo (Google OAuth). Se gestionarán los perfiles personales, datos de contacto y preferencias de uso dentro de la plataforma.

Gestión de Mascotas:

Facilitar el registro, modificación y eliminación de información sobre las mascotas, incluyendo datos clínicos, raza, edad, peso, nivel de actividad y antecedentes médicos. Esta información será utilizada como base para generar recomendaciones nutricionales personalizadas.

Planificación Nutricional:

Generar planes alimenticios equilibrados y adaptados al perfil de cada mascota mediante algoritmos de inteligencia artificial. Los planes considerarán ingredientes disponibles, restricciones alimentarias y objetivos nutricionales definidos por el usuario.

Recetario Colaborativo:

Habilitar un espacio comunitario donde los usuarios puedan crear, compartir y descubrir recetas naturales y saludables. Este módulo fomentará la participación activa de los dueños y la difusión de buenas prácticas alimenticias.

Gestión de Ingredientes y Despensa:

Permitir al usuario registrar los ingredientes disponibles en su hogar ("despensa") para que el sistema genere recomendaciones y recetas compatibles, optimizando el uso de los recursos alimenticios y reduciendo el desperdicio.

Interfaz de Usuario:

Diseñar una interfaz web y móvil intuitiva, accesible y visualmente amigable, orientada tanto a usuarios con experiencia digital como a quienes buscan una herramienta sencilla para mejorar la alimentación de sus mascotas.

Seguridad y Privacidad:

Implementar protocolos de seguridad para proteger la información sensible de los usuarios y sus mascotas. Se establecerán roles, permisos y cifrado de datos, garantizando el acceso únicamente a usuarios autenticados y la confidencialidad de la información.

1.4 Definiciones, acrónimos y abreviaciones

Acrónimo	Descripción
NutriHuella	Plataforma web y móvil desarrollada para promover la alimentación saludable, natural y personalizada para mascotas, mediante el uso de inteligencia artificial.
IA (Inteligencia Artificial)	Conjunto de técnicas computacionales que permiten al sistema analizar datos y generar recomendaciones personalizadas basadas en patrones nutricionales y clínicos.
IA Generativa	Rama de la inteligencia artificial capaz de crear contenido nuevo, como planes de alimentación o recetas naturales, a partir de datos previos.
API (Application Programming Interface)	Interfaz de programación que permite la comunicación entre los módulos del sistema (frontend, backend y servicios externos).
REST API	Estilo arquitectónico utilizado para construir la API del sistema, basado en el protocolo HTTP y operaciones CRUD (Create, Read, Update, Delete).
OAuth (Open Authorization)	Protocolo de autenticación que permite a los usuarios iniciar sesión de forma segura mediante proveedores externos como Google.
JWT (JSON Web Token)	Token de autenticación que contiene información codificada del usuario, usado para autorizar el acceso a los recursos del sistema.
Frontend	Capa visual del sistema desarrollada con Next.js y React, encargada de la interacción directa con el usuario y la presentación de la información.
Backend	Capa lógica desarrollada en Node.js con TypeScript, responsable de procesar solicitudes, aplicar la lógica de negocio y conectarse con la base de datos.
Prisma ORM	Herramienta de mapeo objeto-relacional utilizada para facilitar la interacción entre el backend y la base de datos PostgreSQL.
PostgreSQL	Sistema de gestión de bases de datos relacional utilizado para almacenar información de usuarios, mascotas, planes nutricionales, recetas y preferencias.
Next.js	Framework basado en React que permite renderizado del lado del servidor (SSR) y optimización del rendimiento para aplicaciones web.
TypeScript (TS)	Lenguaje de programación que extiende JavaScript agregando tipado estático, mejorando la escalabilidad y mantenibilidad del código.
Node.js	Entorno de ejecución que permite ejecutar JavaScript en el lado del servidor, facilitando la creación de servicios backend escalables.

Pantry Service (Servicio de Despensa)	Módulo que gestiona los ingredientes disponibles registrados por el usuario y los utiliza para generar recetas compatibles.
Pet Service (Servicio de Mascotas)	Servicio encargado del registro, actualización y consulta de información clínica y de perfil de cada mascota.
Nutrition Service (Servicio Nutricional)	Componente encargado de calcular y generar los planes de alimentación basados en la información del usuario y su mascota.
Recipe Service (Servicio de Recetas)	Módulo dedicado a la creación, almacenamiento y recomendación de recetas naturales dentro del recetario colaborativo.
Auth Service (Servicio de Autenticación)	Módulo que gestiona el registro de usuarios, login, recuperación de contraseña y emisión de tokens JWT.
QA (Quality Assurance)	Conjunto de prácticas y herramientas destinadas a asegurar la calidad del software mediante pruebas funcionales y automatizadas.
BDD (Behavior Driven Development)	Enfoque de desarrollo basado en el comportamiento del usuario final, utilizado junto con la herramienta Behave para pruebas automatizadas.
Allure Report	Herramienta utilizada para generar reportes visuales de los resultados de las pruebas de QA.
CI/CD (Continuous Integration / Continuous Deployment)	Estrategia que automatiza la integración del código, ejecución de pruebas y despliegue del sistema en entornos de desarrollo y producción.
Responsive Design	Diseño adaptable que permite visualizar correctamente la aplicación en distintos dispositivos (computador, tablet y smartphone).
UX/UI (User Experience / User Interface)	Disciplinas orientadas al diseño visual y a la experiencia de uso, buscando una interacción fluida y atractiva para el usuario.
Plan Nutricional Automatizado	Funcionalidad principal de NutriHuella que genera dietas equilibradas según la edad, peso, actividad y preferencias alimenticias de la mascota.

<i>Ficha Digital</i>	<i>Clínica</i>	Registro estructurado que contiene información médica, vacunas y antecedentes relevantes de cada mascota.
<i>Recetario Colaborativo</i>		Espacio dentro de la plataforma donde los usuarios pueden compartir y descubrir recetas naturales creadas por la comunidad.

1.5 Resumen ejecutivo

Este documento de arquitectura de sistema tiene como objetivo proporcionar una descripción general de la arquitectura propuesta para la plataforma **NutriHuella**, una solución tecnológica orientada al bienestar y la nutrición saludable de las mascotas mediante el uso de **inteligencia artificial**.

El presente documento forma parte del proceso de **diseño y planificación arquitectónica**, y busca establecer una base sólida para el desarrollo, implementación y mantenimiento del sistema. La arquitectura propuesta contempla una estructura modular, escalable y segura, que integra distintos servicios como la **gestión de usuarios y mascotas**, la **ficha clínica digital**, el **plan nutricional automatizado** y el **recetario colaborativo**.

A través de este diseño, **NutriHuella** busca ofrecer una herramienta accesible e innovadora que permita a los dueños de mascotas **mejorar la alimentación y calidad de vida de sus animales**, promoviendo prácticas responsables y saludables respaldadas por tecnología.

1.6 Representación

La arquitectura del sistema **NutriHuella** ha sido diseñada basándose en el enfoque del **marco de trabajo 4+1** propuesto por Philippe Kruchten, complementado con las buenas prácticas del **proceso unificado de desarrollo de software (RUP)**. Este modelo permite representar el sistema desde diferentes perspectivas que abarcan tanto la estructura lógica como los procesos, la implementación y los escenarios de interacción con los usuarios.

Las perspectivas consideradas en esta versión del documento abarcan:

- **Vista Lógica del Sistema:**
Profundiza en la estructura interna del sistema, mostrando los principales módulos que conforman NutriHuella. En esta vista se representan los **diagramas de clases**, el **modelo entidad-relación (E-R)** y los **diagramas de secuencia**, los cuales reflejan las relaciones entre los servicios de autenticación, gestión de mascotas, motor nutricional y recetario colaborativo. Su propósito es describir la organización del software desde una perspectiva funcional y de diseño.
- **Vista de Despliegue / Componentes:**
Presenta los **diagramas de componentes y de paquetes**, donde se detalla la organización modular del sistema, sus dependencias y los niveles de abstracción entre el **frontend**, el **backend** y la **base de datos**. Esta vista evidencia cómo los distintos módulos se integran dentro del entorno de ejecución, asegurando una arquitectura escalable, modular y mantenible.
- **Vista de Procesos:**
Describe los **flujos de actividad** y la interacción entre los diferentes procesos del sistema. A través del **diagrama de actividad**, se representa el funcionamiento dinámico de NutriHuella, incluyendo la comunicación entre servicios como el registro de usuarios, la creación de fichas clínicas, la generación de planes nutricionales automatizados y la gestión del recetario.

- **Vista Física:**
Expone la **infraestructura de despliegue** del sistema mediante el **diagrama de despliegue**, mostrando los entornos de desarrollo, prueba y producción. Incluye la configuración de servidores, contenedores **Docker**, servicios en la nube (por ejemplo, **Vercel** o **AWS**) y la conexión con la base de datos **PostgreSQL**, garantizando la disponibilidad y seguridad de la aplicación.
- **Vista de Escenarios / Casos de Uso:**
Detalla los **casos de uso principales** de la plataforma, mostrando los actores involucrados (usuario, sistema, motor de IA, proveedor de autenticación, etc.) y las interacciones con los distintos módulos. Esta vista también presenta los **escenarios de calidad**, donde se evalúan atributos arquitectónicos como el rendimiento, la seguridad, la usabilidad y la escalabilidad del sistema.

2 Metas y Restricciones de la Arquitectura

2.1 Metas de la arquitectura

La arquitectura del sistema **NutriHuella** se ha diseñado con el propósito de establecer una base sólida, escalable y segura que permita el desarrollo de una plataforma web y móvil enfocada en la **nutrición personalizada y el bienestar de las mascotas**.

Las metas de la arquitectura buscan garantizar la **calidad técnica**, la **eficiencia operativa** y la **alineación con los objetivos funcionales** del proyecto.

A continuación, se detallan las principales metas:

- **Escalabilidad:**
Permitir que el sistema soporte un número creciente de usuarios, mascotas y recetas sin afectar el rendimiento. La arquitectura modular y la separación por servicios facilitan la incorporación de nuevas funcionalidades en el futuro.
- **Seguridad y Privacidad:**
Proteger la información sensible de los usuarios y sus mascotas mediante protocolos de autenticación seguros (**OAuth 2.0** y **JWT**) y buenas prácticas de manejo de datos. Se busca asegurar la confidencialidad, integridad y disponibilidad de la información.
- **Disponibilidad y Fiabilidad:**
Garantizar que la plataforma esté disponible de forma continua a través de un entorno web estable, con mecanismos de respaldo y monitoreo que reduzcan los riesgos de interrupciones o pérdida de datos.
- **Usabilidad y Accesibilidad:**
Ofrecer una interfaz intuitiva, moderna y adaptable a diferentes dispositivos

(computador, tablet y smartphone), asegurando una experiencia de uso fluida, accesible y amigable para todo tipo de usuarios.

- **Mantenibilidad:**

Facilitar la evolución del sistema a través de un código ordenado, documentado y estructurado por capas (frontend, backend y base de datos), lo que permite una rápida detección y corrección de errores, así como la integración de nuevas funciones.

- **Integración con Servicios Inteligentes:**

Incorporar componentes de **inteligencia artificial** que permitan generar planes alimenticios personalizados y recomendaciones basadas en datos clínicos, ingredientes disponibles y preferencias de cada mascota.

- **Calidad del Software:**

Asegurar la validación del sistema mediante **pruebas automatizadas con Selenium y Gherkin**, además de **pruebas manuales funcionales**, garantizando que las distintas funcionalidades cumplan con los requerimientos establecidos y que la plataforma mantenga altos estándares de calidad.

2.2 Restricciones de la Arquitectura

La arquitectura de **NutriHuella** se ha definido considerando un conjunto de restricciones técnicas, funcionales y operativas que condicionan el alcance y el diseño del sistema. Estas limitaciones buscan mantener la viabilidad del proyecto dentro de los recursos, plazos y tecnologías disponibles, asegurando al mismo tiempo la calidad y estabilidad de la solución.

A continuación, se detallan las principales restricciones identificadas:

- **Base de datos inicial limitada:**
La plataforma comenzará con una base de datos reducida de alimentos, nutrientes y restricciones alimenticias solo para perro. Su expansión requerirá actualizaciones manuales y validación continua por parte del equipo de desarrollo o nutricionistas colaboradores.
- **Dependencia de la información del usuario:**
La precisión de las recomendaciones nutricionales generadas por la inteligencia artificial depende directamente de la **veracidad y completitud de los datos ingresados por el usuario** (como peso, edad, alergias o nivel de actividad de la mascota).
- **No reemplaza el diagnóstico profesional:**
NutriHuella actúa como una herramienta de apoyo informativo y educativo. No sustituye la evaluación ni el diagnóstico realizado por un veterinario profesional, sino que busca **complementar** la toma de decisiones del dueño responsable.
- **Cobertura geográfica inicial:**
La aplicación se encuentra inicialmente enfocada en el contexto chileno, con recetas y recomendaciones basadas en **ingredientes y productos disponibles en Chile**. Su expansión internacional requerirá ajustes en las bases de datos de alimentos y unidades de medida.
- **Recursos tecnológicos y financieros limitados:**
El proyecto se desarrolla dentro del marco académico del portafolio de título, por lo que su infraestructura depende de servicios gratuitos o de bajo costo. Esto impone restricciones en escalabilidad, capacidad de almacenamiento y herramientas de monitoreo.
- **Pruebas de validación:**
El aseguramiento de calidad se realizará mediante **pruebas automatizadas con Selenium y Gherkin**, complementadas con **pruebas manuales funcionales**. Esto implica que la cobertura de pruebas podría ser parcial en las primeras versiones del sistema.
- **Integración futura con servicios externos:**
Aunque la arquitectura considera una base sólida para futuras integraciones (como servicios veterinarios), dichas funcionalidades no serán implementadas en la versión inicial del sistema.

2.3 Reglas de Negocio

A. Principios transversales

BN-01 Todo recurso ligado a un usuario (Pet, PantryItem, Recipe, FavoriteRecipe, RecipeFeedback, NutritionProfile(History), RecipeGen*) solo puede ser gestionado por su propietario.

BN-02 Los catálogos (Species, SexType, SizeType, DietType, ActivityLevel, GoalType, DifficultyType, IngredientCategory, DiseaseStatus, UnitType, VaccinationType, DiseaseType, UserRole) son cerrados: siempre se elige desde lista.

BN-03 Las entidades gestionadas por usuarios registran auditoría mínima: createdAt y, cuando aplique, updatedAt.

BN-04 Las fechas visibles al usuario se interpretan en su zona horaria y se almacenan en ISO-8601.

BN-05 Los identificadores de negocio son de tipo texto (compatibles con UUID) y no se reutilizan.

BN-06 Las respuestas de colecciones deben ser paginadas.

BN-07 Operaciones propensas a duplicados (p. ej., marcar favorito) deben ser idempotentes.

B. Usuarios y roles

BN-08 User.email es único y obligatorio.

BN-09 User.roleId debe existir en UserRole.

BN-10 Si existe googleId, es único y habilita inicio de sesión social; el proveedor autentica y la autorización es interna.

C. Mascotas, especies y razas

BN-11 Alta de mascota requiere name, speciesId, sexId, sizeId, birthDate, weightKg.

BN-12 breedId y breedOther son excluyentes (una u otra, nunca ambas).

BN-13 Pet.weightKg siempre refleja la medición vigente más reciente del histórico.

BN-14 Una mascota solo puede existir si pertenece a un usuario (no hay mascotas huérfanas).

BN-15 El nombre de la mascota es obligatorio y editable sin unicidad global.

D. Clínica (registro clínico, vacunas, enfermedades, pesos)

BN-16 ClinicalRecord es 0..1 por Pet; si existe, se actualiza, no se duplica.

BN-17 Cada vacuna se registra como evento histórico en Vaccination.

BN-18 Cada enfermedad se registra como evento en Disease con diseaseId y statusId.

BN-19 Estados de enfermedad válidos: ACTIVE, RESOLVED, CRONIC.

BN-20 Para la misma enfermedad y fecha no pueden coexistir estados incompatibles.

BN-21 VaccinationType y DiseaseType deben existir en catálogo.

BN-22 WeightHistory es la fuente de verdad del peso.

BN-23 Los pesos deben ser positivos y razonables para especie/tamaño.

BN-24 Para una misma fecha solo queda una medición vigente; otra del mismo día la reemplaza o se rechaza.

BN-25 Al borrar la medición más reciente, se recalcula Pet.weightKg con la siguiente por fecha.

E. Nutrición (perfil e histórico)

BN-26 Cada Pet tiene a lo sumo un NutritionProfile activo (0..1).

BN-27 dietTypeId, activityLevelId, goalId deben existir en sus catálogos.

BN-28 Cualquier cambio al perfil crea una versión en NutritionProfileHistory (no se sobrescribe el perfil).

BN-29 La versión se calcula como MAX(version)+1 por profileId, registrando changedAt y changedByUserId.

BN-30 Las versiones de NutritionProfileHistory son inmutables.

BN-31 Si dailyCalories/waterIntakeMl no se informan, pueden derivarse de peso, actividad y objetivo; el valor final se persiste en el perfil y en la versión.

BN-32 El perfil puede existir con campos nulos válidos según UX (p. ej., alergias).

F. Despensa (ingredientes)

BN-33 PantryItem pertenece al usuario (no a la mascota).

BN-34 PantryItem.name es obligatorio; normalized se genera desde name para búsqueda y deduplicación.

BN-35 unitId y categoryId (si se informan) deben existir en UnitType e IngredientCategory.

BN-36 quantityBase expresa la conversión de quantity a una unidad de referencia para comparaciones.

BN-37 Estados por vencimiento: Vigente (sin fecha o > hoy), Por vencer (hoy..hoy+7), Vencido (< hoy).

BN-38 Listados “por vencer” se ordenan por expiresAt ascendente; ítems sin fecha no participan en alertas.

G. Recetas, favoritos y feedback

BN-39 Recipe requiere title y content en JSON válido (ingredientes, pasos, porciones mínimas).

BN-40 Recipe.userId es obligatorio; petId es opcional (general o asociada a mascota).

BN-41 difficultyId —si se envía— debe existir en DifficultyType; sin valor se considera “Sin dificultad”.

BN-42 FavoriteRecipe es única por (userId, recipeId); petId opcional agrega contexto.

BN-43 Marcar/Desmarcar favorito es idempotente.

BN-44 RecipeFeedback.rating es entero 1..5; una nueva valoración del mismo usuario para la misma receta reemplaza la anterior.

BN-45 Al eliminar una receta se resuelven sus referencias en favoritos y valoraciones (no deben quedar huérfanos).

H. IA: generación y límites

BN-46 Cada solicitud de generación se registra en RecipeGenAttempt como evento no editable.

BN-47 Solo las generaciones aceptadas/guardadas se registran en RecipeGeneration.

BN-48 Puede aplicarse una política de límites de intentos por día/usuario/mascota; al

superarse, se rechaza con mensaje claro.

I. Integridad, orden y concurrencia

BN-49 Orden natural recomendado: Recipe por createdAt desc; WeightHistory por date desc; PantryItem por estado de vencimiento y expiresAt asc; NutritionProfileHistory por version asc.

BN-50 Operaciones multi-paso con riesgo de carrera (p. ej., editar perfil, moderar) deben resolver concurrencia simple rechazando cambios si updatedAt cambió.

J. Membresía y límites

BN-51 Plan del usuario: User.plan $\in$ {FREE, PREMIUM}; al cambiar el plan se actualiza membershipUpdatedAt.

BN-52 Límite de mascotas por plan: no crear si count(Pet.ownerId) $\geq$ MAX_PETS(plan).

BN-53 Cuota diaria de generación (IA): no iniciar si attempts_today(userId) $\geq$ DAILY_GEN_LIMIT(plan).

BN-54 Cómputo de intentos: attempts_today(userId) = count(RecipeGenAttempt en [inicio_día_local, fin_día_local]).

BN-55 Reset diario: la cuota se reinicia al inicio del día local; no se persisten contadores (se deriva de RecipeGenAttempt).

BN-56 Features premium: acciones marcadas como “premium” requieren User.plan = PREMIUM.

BN-57 Cambio de plan: actualiza membershipUpdatedAt; no altera recursos existentes, aplica a nuevas operaciones.

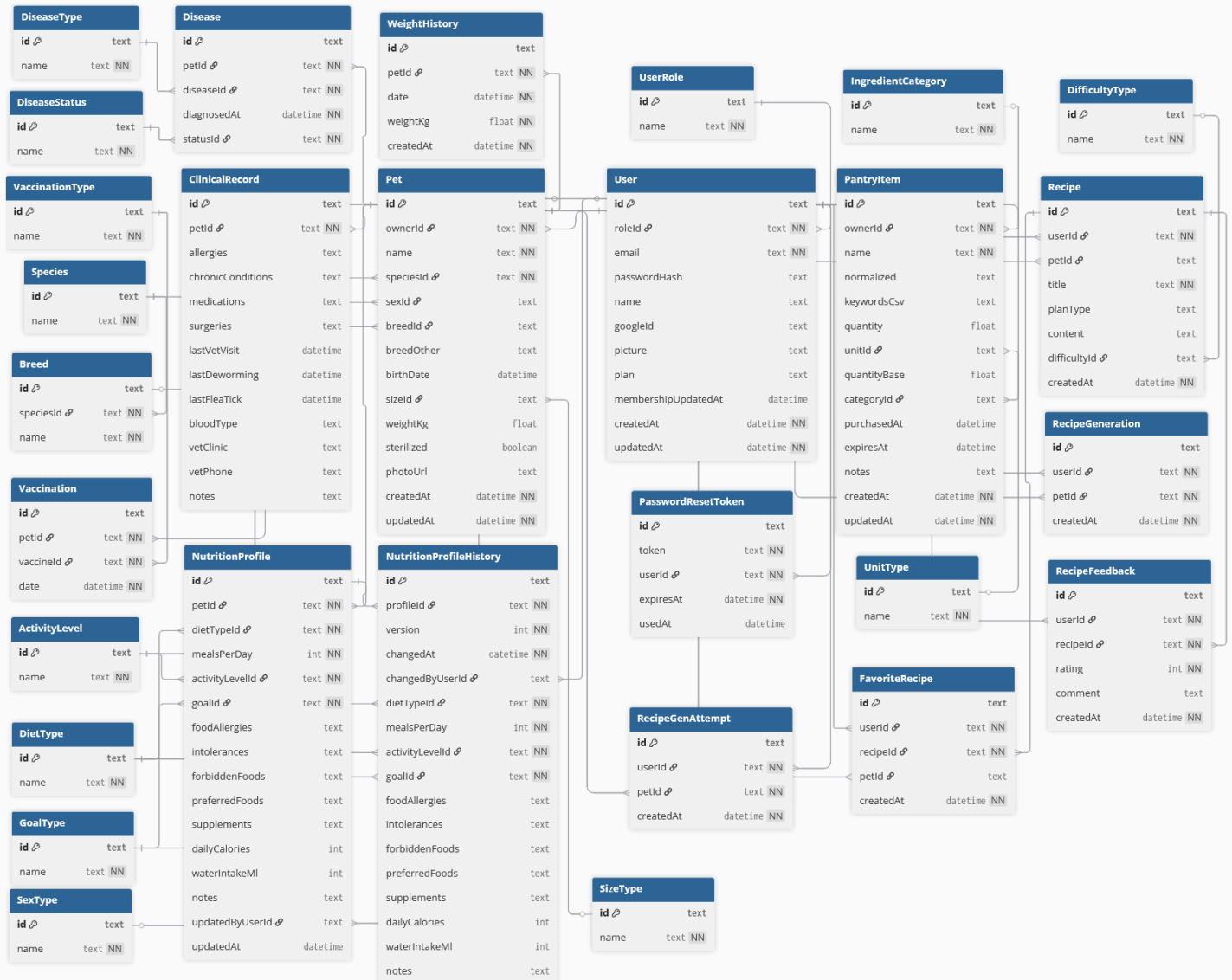
BN-58 Degradación a FREE: si supera límites de FREE, conserva lo existente pero no puede crear más ni usar features premium.

BN-59 Mensajería de límites: al bloquear por plan/límite, incluir motivo y valores (p. ej., MAX_PETS, DAILY_GEN_LIMIT) y llamada a acción.

3 Modelo 4+1

3.1 Vista lógica

Modelo E-R



Descripción del modelo E-R

El esquema se organiza por dominios: **Usuarios** (User, UserRole), **Mascotas** (Pet, Species, Breed, SexType, SizeType), **Clínica** (ClinicalRecord, Vaccination/VaccinationType, Disease/DiseaseType/DiseaseStatus, WeightHistory), **Nutrición** (NutritionProfile y su histórico versionado NutritionProfileHistory con catálogos DietType, ActivityLevel, GoalType), **Despensa** (PantryItem con UnitType e IngredientCategory), **Recetas** (Recipe con DifficultyType, más FavoriteRecipe y RecipeFeedback), **Seguridad** (PasswordResetToken) e **IA** (RecipeGenAttempt, RecipeGeneration).

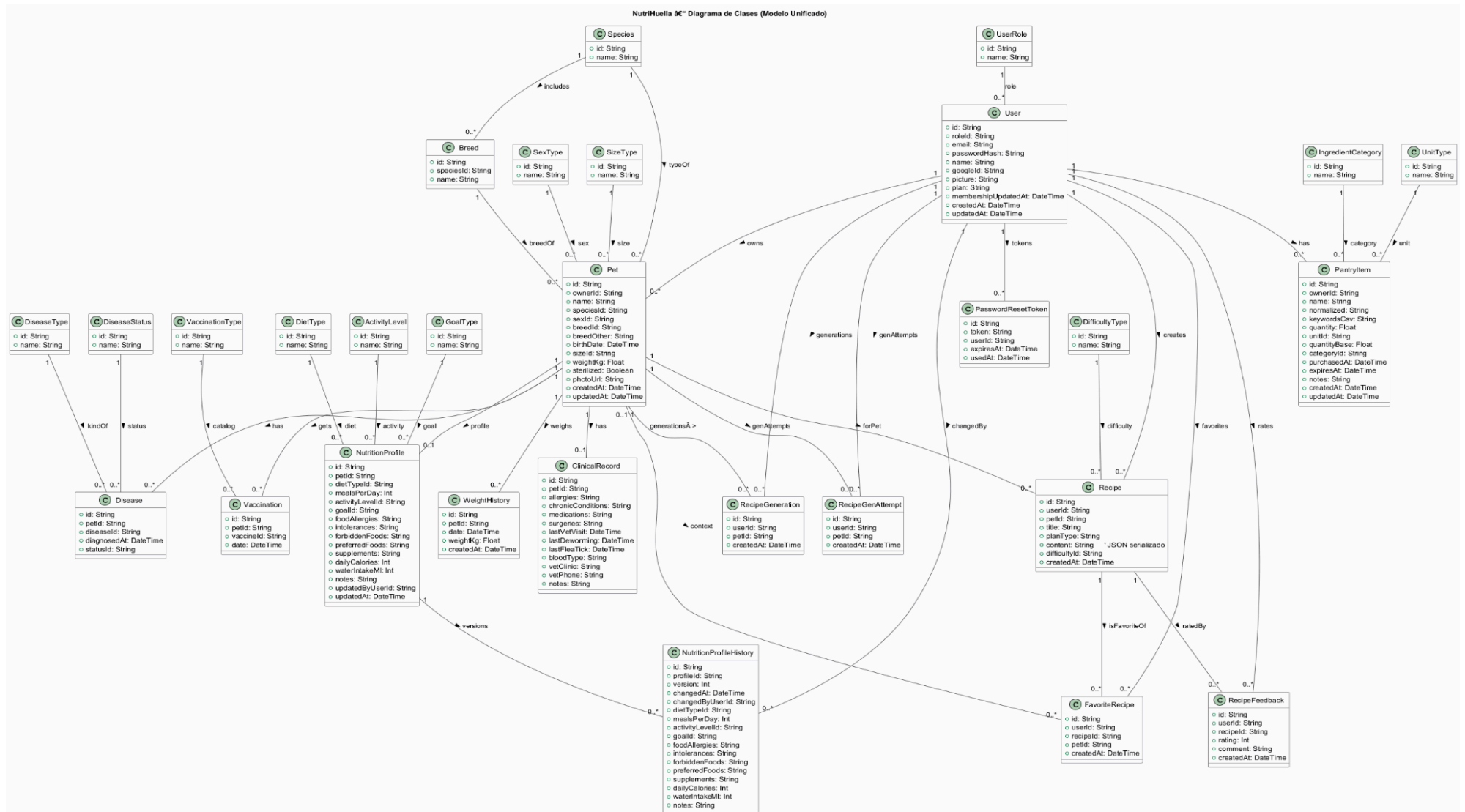
Relaciones clave

- User 1—* Pet y User 1—* PantryItem.
- Pet 1—1 ClinicalRecord y Pet 1—1 NutritionProfile.
- Pet 1—* WeightHistory, Vaccination y Disease.
- NutritionProfile 1—* NutritionProfileHistory (versiones inmutables por profileId, version).
- User 1—* Recipe; DifficultyType 1—* Recipe.
- Tablas puente: FavoriteRecipe (única por userId, recipId) y RecipeFeedback.

Criterios de diseño

Catálogos normalizados para consistencia y filtrado, identificadores de tipo texto y fechas ISO-8601. Índices en consultas frecuentes (p. ej., (petId, date) en WeightHistory, ownerId/ExpiresAt en PantryItem) y unicidades donde corresponde (profileId, version y userId, recipId). El modelo prioriza coherencia del dominio, trazabilidad (históricos y versionado) y un acceso eficiente a la información cotidiana del usuario.

Diagrama de Clase



Descripción del Diagrama de Clases (NutriHuella)

El modelo se organiza en cuatro áreas:

1) Catálogos (lookups)

UserRole, Species, SexType, SizeType, DietType, ActivityLevel, GoalType, DifficultyType, IngredientCategory, DiseaseStatus, UnitType, VaccinationType, DiseaseType. Centralizan valores controlados y se relacionan 1→* con las entidades que los usan (p. ej., DifficultyType → Recipe, UnitType/IngredientCategory → PantryItem).

2) Núcleo de dominio

User (raíz de cuenta) posee Pet y PantryItem. Breed depende de Species. Pet referencia especie/sexo/tamaño/raza o breedOther. Entidades con auditoría (createdAt, y updatedAt donde aplica).

3) Clínica, Nutrición y Despensa

- Clínica: Pet 1→0..1 ClinicalRecord; 1→* Vaccination (VaccinationType) y Disease (DiseaseType, DiseaseStatus).
- Nutrición: Pet 1→0..1 NutritionProfile; 1→* NutritionProfileHistory (versionado con changedByUserId, version).
- Despensa: PantryItem pertenece al **usuario**; incluye normalized, unitId y categoryId.

4) Recetas, Favoritos, Feedback e IA

Recipe (creada por User, opcional Pet) usa DifficultyType y content JSON. N:M con clases dedicadas: FavoriteRecipe (única por userId+recipeId, petId opcional) y RecipeFeedback (rating 1–5). Eventos: RecipeGenAttempt (intentos) y RecipeGeneration (resultados guardados).

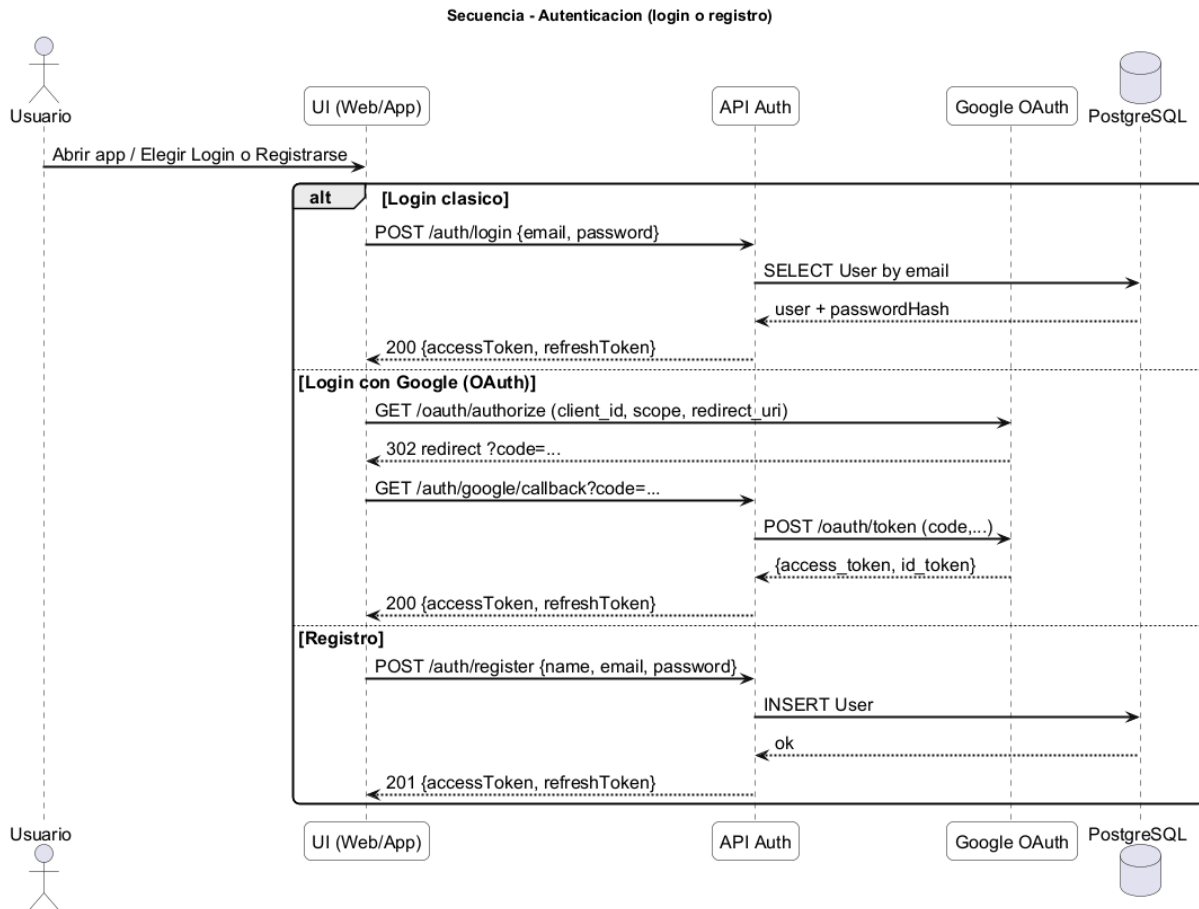
Cardinalidades clave (selección)

User 1→* Pet, User 1→* PantryItem, User 1→* Recipe; Pet 1→0..1 ClinicalRecord y NutritionProfile; Pet 1→* WeightHistory/Vaccination/Disease/Recipe; NutritionProfile 1→* NutritionProfileHistory.

Notas de diseño

Catálogos para validación y filtrado, normalización de ingredientes para búsqueda/deduplicación, versionado explícito en nutrición y uso consistente de IDs de texto y fechas ISO-8601.

Diagrama de Secuencia



Este diagrama representa el **flujo de autenticación en NutriHuella**, mostrando cómo el **usuario** interactúa con la **interfaz web o móvil (UI)** y los **servicios de autenticación (API Auth, Google OAuth y base de datos PostgreSQL)**.

El proceso contempla tres caminos posibles:

1. **Login clásico:**

El usuario ingresa su correo y contraseña.

La API consulta en la base de datos (**SELECT User by email**) y, si las credenciales son correctas, devuelve los tokens de sesión (**accessToken, refreshToken**).

2. **Login con Google (OAuth):**

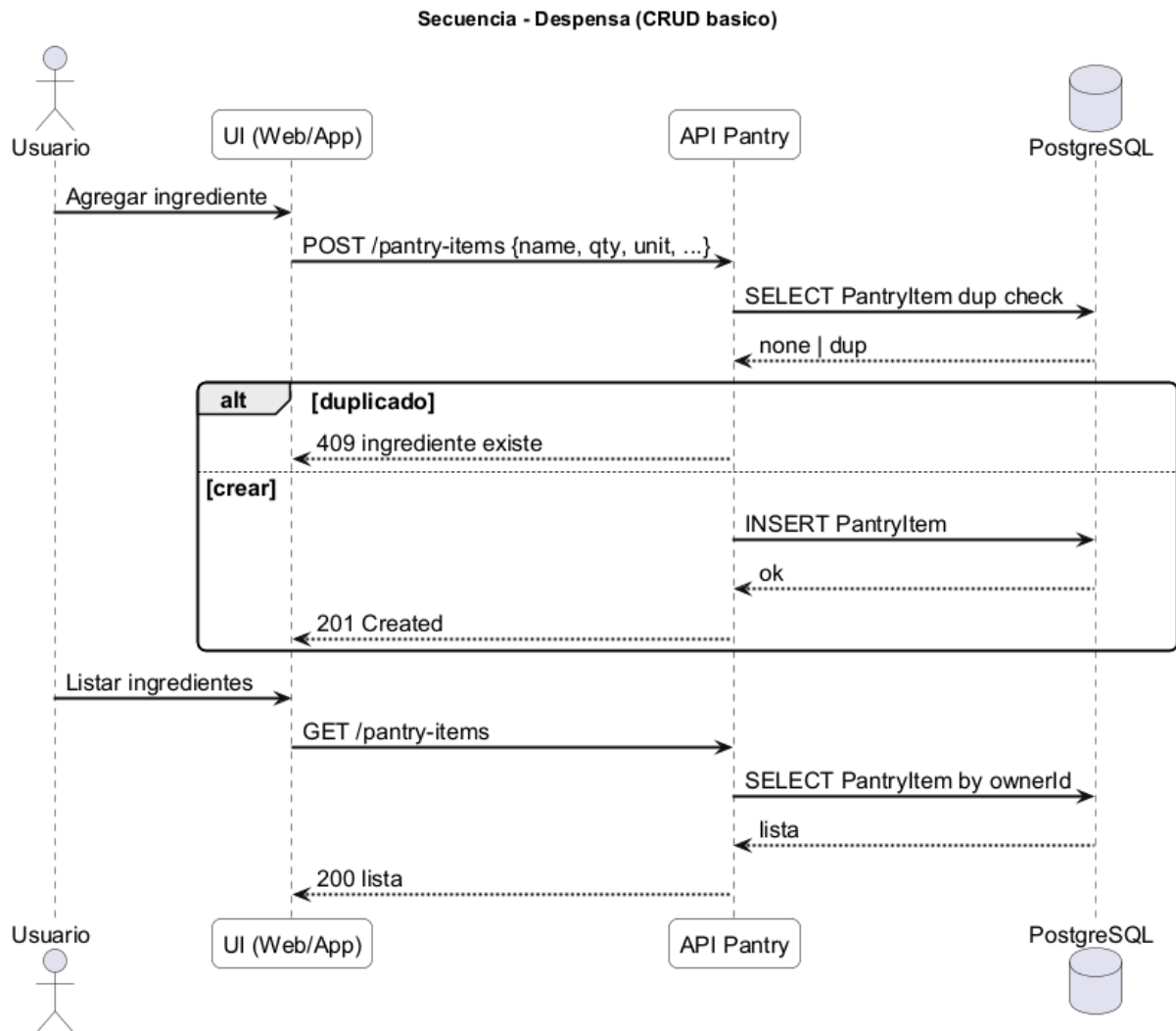
El usuario es redirigido al proveedor de identidad (**Google OAuth**) para autorizar el acceso.

Tras la validación, el sistema obtiene los tokens de Google y los intercambia por sus propios tokens de sesión (**POST /oauth/token**).

3. **Registro:**

El usuario completa sus datos básicos (nombre, correo, contraseña).

La API registra la cuenta en la base de datos (**INSERT User**) y responde con un **accessToken** y **refreshToken**, iniciando sesión automáticamente.



Este diagrama muestra el proceso de **registro de una mascota** en NutriHuella, desde que el **usuario** ingresa los datos hasta que la información queda almacenada en la base de datos.

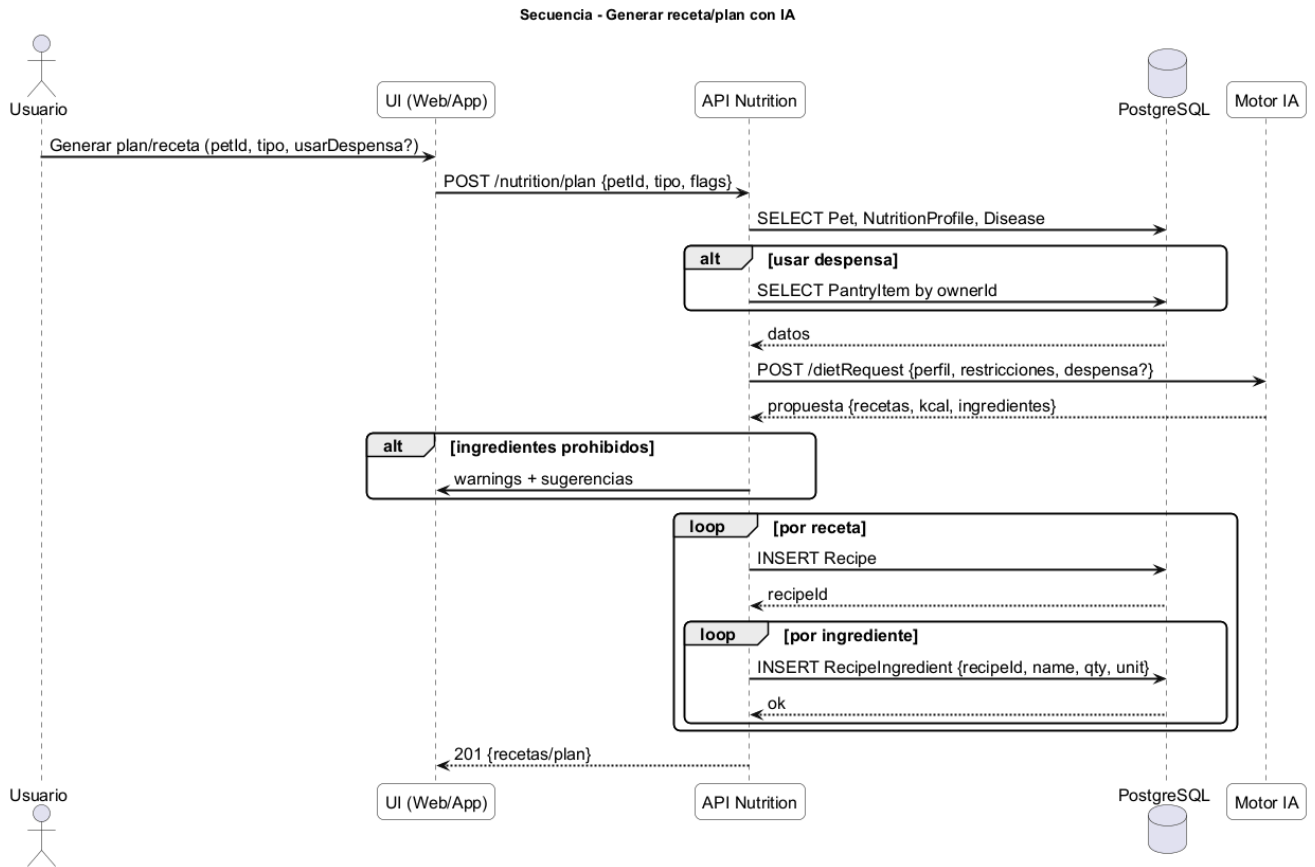
El flujo inicia cuando el usuario selecciona “**Registrar Mascota**” en la interfaz.

La **API de Mascotas (API Pets)** verifica que no exista un registro duplicado para el mismo propietario, nombre y especie.

Si no existe, se crea la mascota (**INSERT Pet**) y automáticamente se genera su **perfil nutricional inicial (INSERT NutritionProfile)**.

Opcionalmente, el usuario puede agregar el **peso inicial** y una **foto de la mascota**; la imagen se guarda en el **Object Storage** y su URL se asocia al registro en la base de datos (**UPDATE Pet SET photoUrl**).

Finalmente, el sistema confirma la creación con un **código 201 (petId)**, mostrando la mascota registrada al usuario.



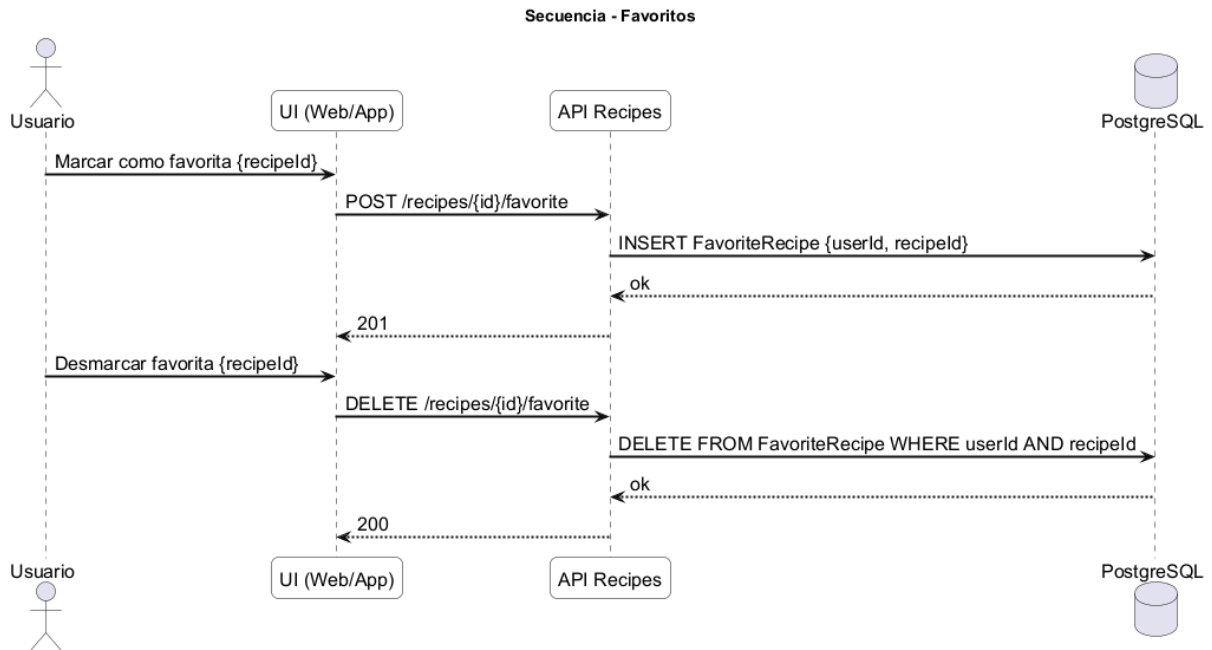
Este diagrama muestra el proceso mediante el cual el **usuario** solicita la generación de una **receta o plan nutricional personalizado** para su mascota.

El flujo comienza cuando el usuario elige la opción “**Generar plan/receta**” en la interfaz. La **API Nutrition** obtiene la información del perfil nutricional y condiciones médicas de la mascota desde la base de datos (**PostgreSQL**). En caso de no estar completado el perfil de la mascota, aparecerá un warning.

Si el usuario decide usar los ingredientes disponibles en su despensa, la API también recupera esos datos.

Luego, la API envía una solicitud al **Motor de IA**, que genera una propuesta de recetas con su valor calórico e ingredientes recomendados. Si se detectan **ingredientes prohibidos**, el sistema muestra advertencias y sugerencias.

Finalmente, las recetas se almacenan en la base de datos (**INSERT Recipe** y **INSERT RecipeIngredient**) y se devuelve una confirmación **201 (recetas/plan)** al usuario.

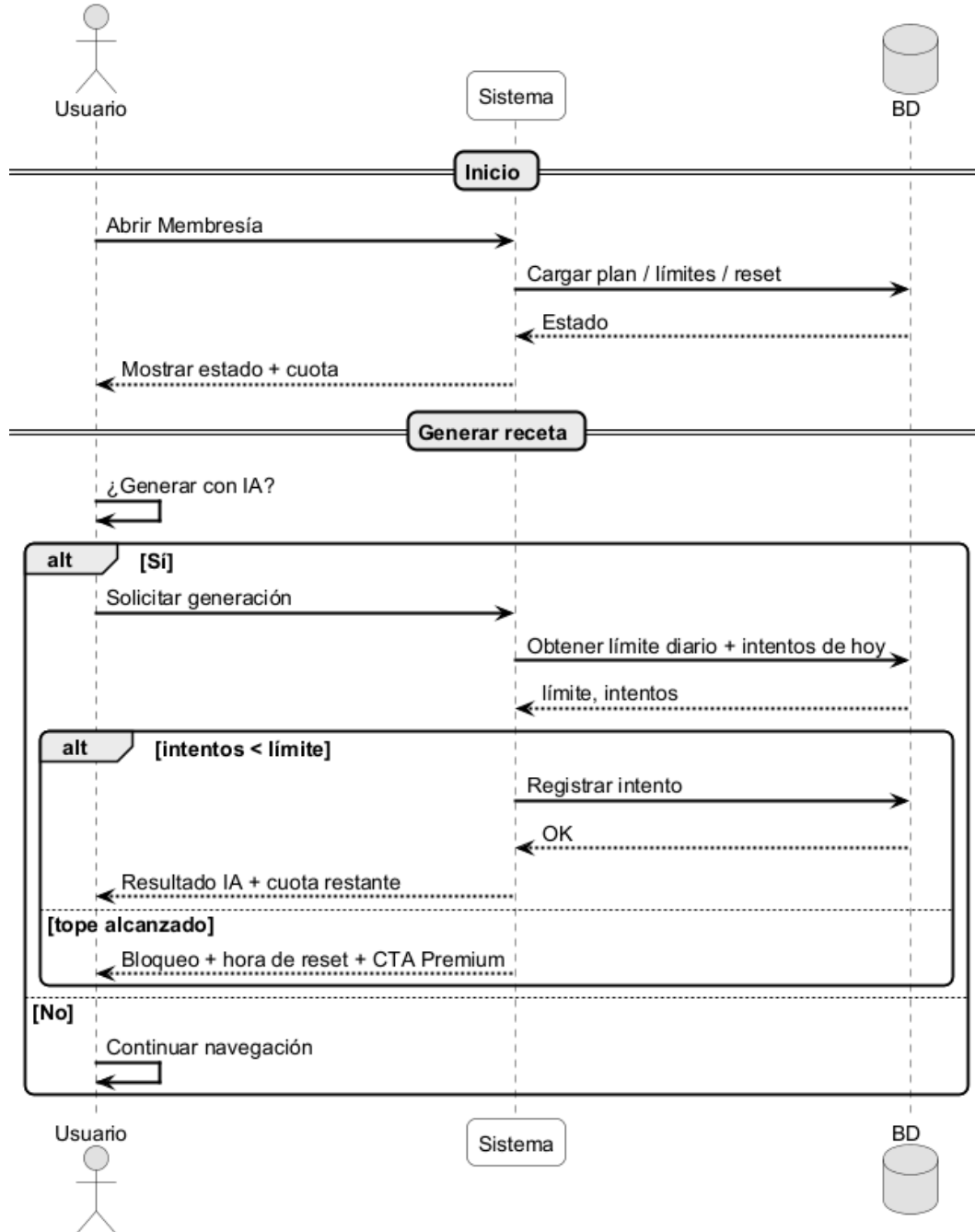


Este diagrama representa el flujo mediante el cual el **usuario** puede **marcar o desmarcar una receta como favorita** dentro del sistema NutriHuella.

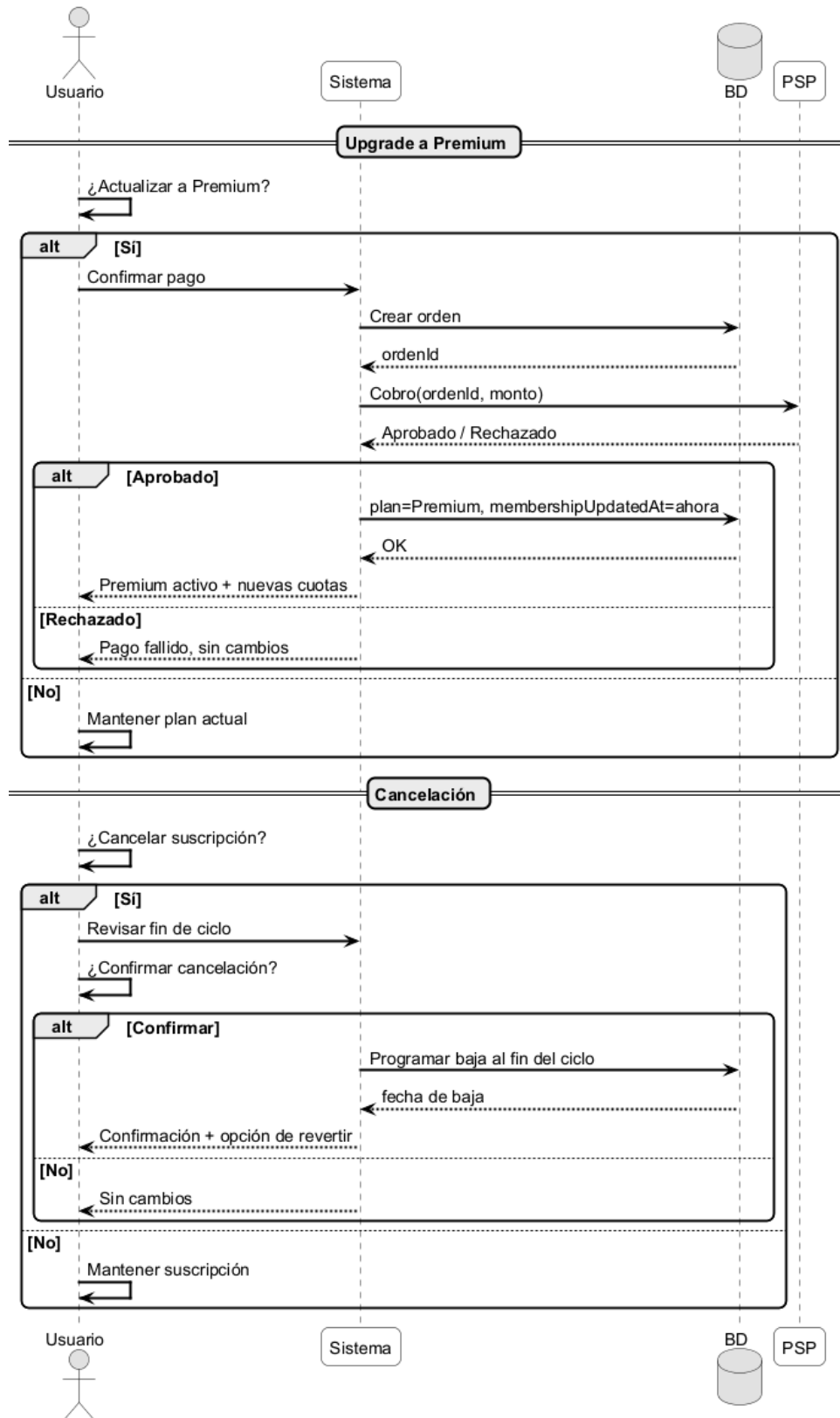
El proceso comienza cuando el usuario selecciona la opción **“Marcar como favorita”**. La interfaz **(UI Web/App)** envía la solicitud a la **API de Recetas**, que registra la acción en la base de datos (**INSERT FavoriteRecipe**) asociando el identificador del usuario y la receta. El sistema confirma la operación con una respuesta **201 (creado)**.

Si el usuario decide **quitar la receta de favoritos**, la API recibe una solicitud **DELETE**, elimina el registro correspondiente (**DELETE FROM FavoriteRecipe WHERE userId AND recipeld**) y devuelve una confirmación **200 (ok)**.

Membresía Premium — Parte 1 (Inicio + Generar Receta)



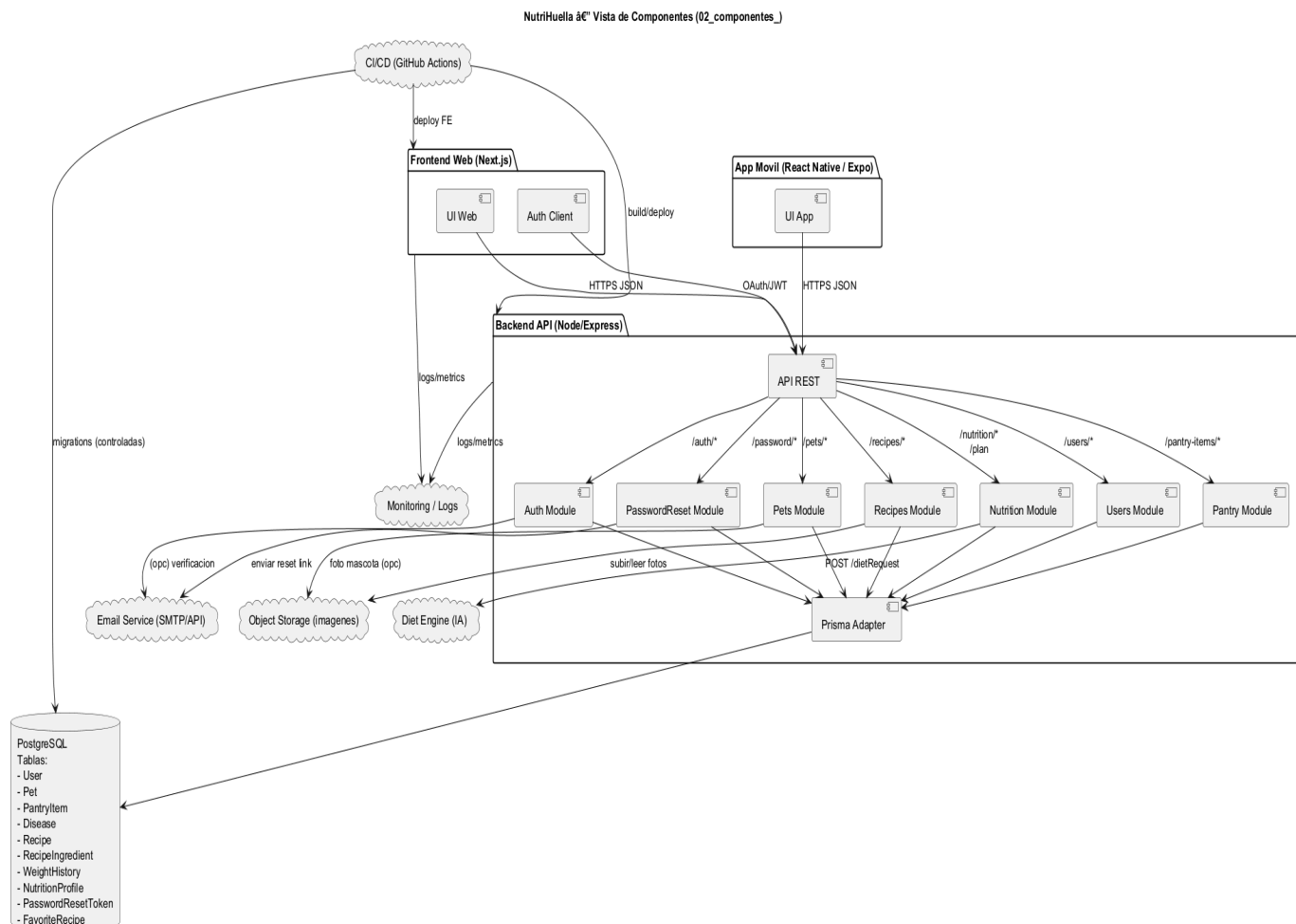
Membresía Premium — Parte 2 (Upgrade + Cancelación)



En este diagrama en dos partes, primero, al acceder a **Membresía**, la **UI** consulta en la **BD** el **plan (FREE/PREMIUM)**, la **cuota diaria** disponible y la **hora de reinicio**; acto seguido, muestra ese estado al usuario. Si el usuario elige “**Generar receta con IA**”, entonces la **API** verifica en la **BD** el **límite diario** y el conteo de **intentos** del día: si aún hay **disponibilidad**, registra un nuevo **intento**, ejecuta la **generación con IA** y devuelve un **borrador** junto con la **cuota actualizada**; de lo contrario, la interfaz informa el **bloqueo**, expone la hora exacta del próximo **reset** e invita a considerar el **upgrade a Premium**. A continuación, y como continuación natural del flujo anterior, el usuario puede iniciar el “**Upgrade a Premium**”: tras su **confirmación**, el sistema crea una **orden** en la **BD** y la envía al **PSP**; si el **pago aprobado**, el plan del usuario se actualiza a **PREMIUM** (con su **marca temporal**), y la **UI** refleja los **beneficios** y las **nuevas cuotas**; si el **pago rechazado**, se notifica el fallo y el plan queda sin cambios. Finalmente, para la **cancelación**, el usuario revisa el **fin de ciclo** vigente y, al confirmar, el sistema **programa la baja** al término del ciclo; en consecuencia, la **interfaz** muestra la **fecha exacta** y ofrece la opción de **revertir** antes de que la cancelación se haga efectiva.

3.2 Vista Despliegue

Diagrama de componentes de paquetes



Este diagrama muestra la arquitectura lógica y los principales **componentes de software** que conforman NutriHuella, organizados según su función dentro del sistema.

El **Frontend Web (Next.js)** y la **App Móvil (React Native / Expo)** actúan como clientes de usuario, comunicándose mediante **API REST (HTTPS JSON)** con el **Backend API (Node/Express)**, que centraliza la lógica del sistema.

El backend está dividido en **módulos especializados**:

- **Auth Module:** autenticación y emisión de tokens JWT/OAuth.
- **PasswordReset Module:** gestión de recuperación de contraseñas.
- **Pets Module:** manejo de información de mascotas y perfiles nutricionales.
- **Recipes Module:** generación y gestión de recetas.

- **Nutrition Module:** conexión con el motor de IA para planes nutricionales.
- **Users y Pantry Modules:** administración de usuarios e ingredientes.

El sistema utiliza **Prisma Adapter** para la comunicación con la base de datos **PostgreSQL**, que almacena entidades clave como usuarios, recetas, mascotas, historial de peso y favoritos.

Servicios complementarios incluyen **Email Service** (verificación y reset), **Object Storage** (imágenes) y **Diet Engine (IA)**.

Finalmente, el sistema incorpora **monitoreo y logs** automáticos y un flujo de **CI/CD (GitHub Actions)** para despliegue continuo del frontend y backend.

3.3 Vista de Procesos

Diagrama de actividad

A continuación, se presentan los **Diagramas de Actividad** correspondientes a cinco procesos fundamentales del sistema **NutriHuella**:

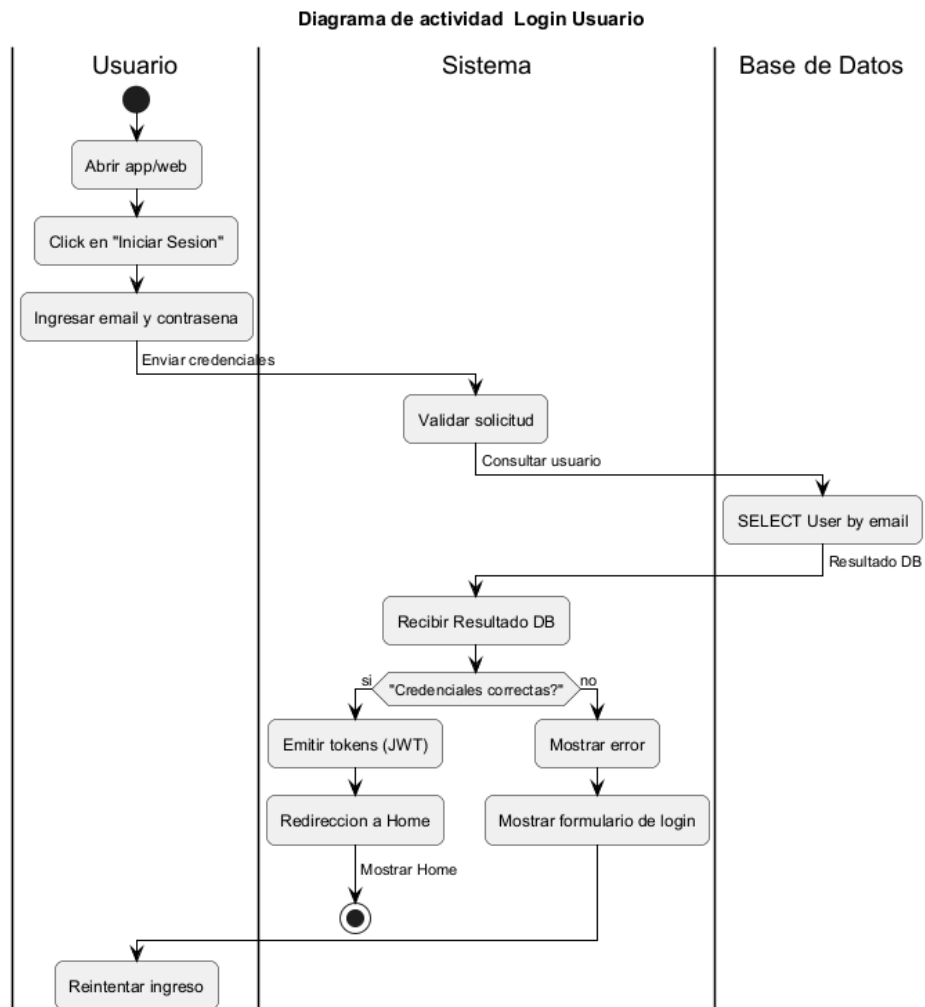
Login, Recuperar Contraseña, Registrar Mascota, Generar Receta/Plan con IA y Membresía Premium.

Estos diagramas muestran el flujo de acciones y decisiones que realiza el usuario dentro de la aplicación, así como las interacciones con los servicios del sistema.

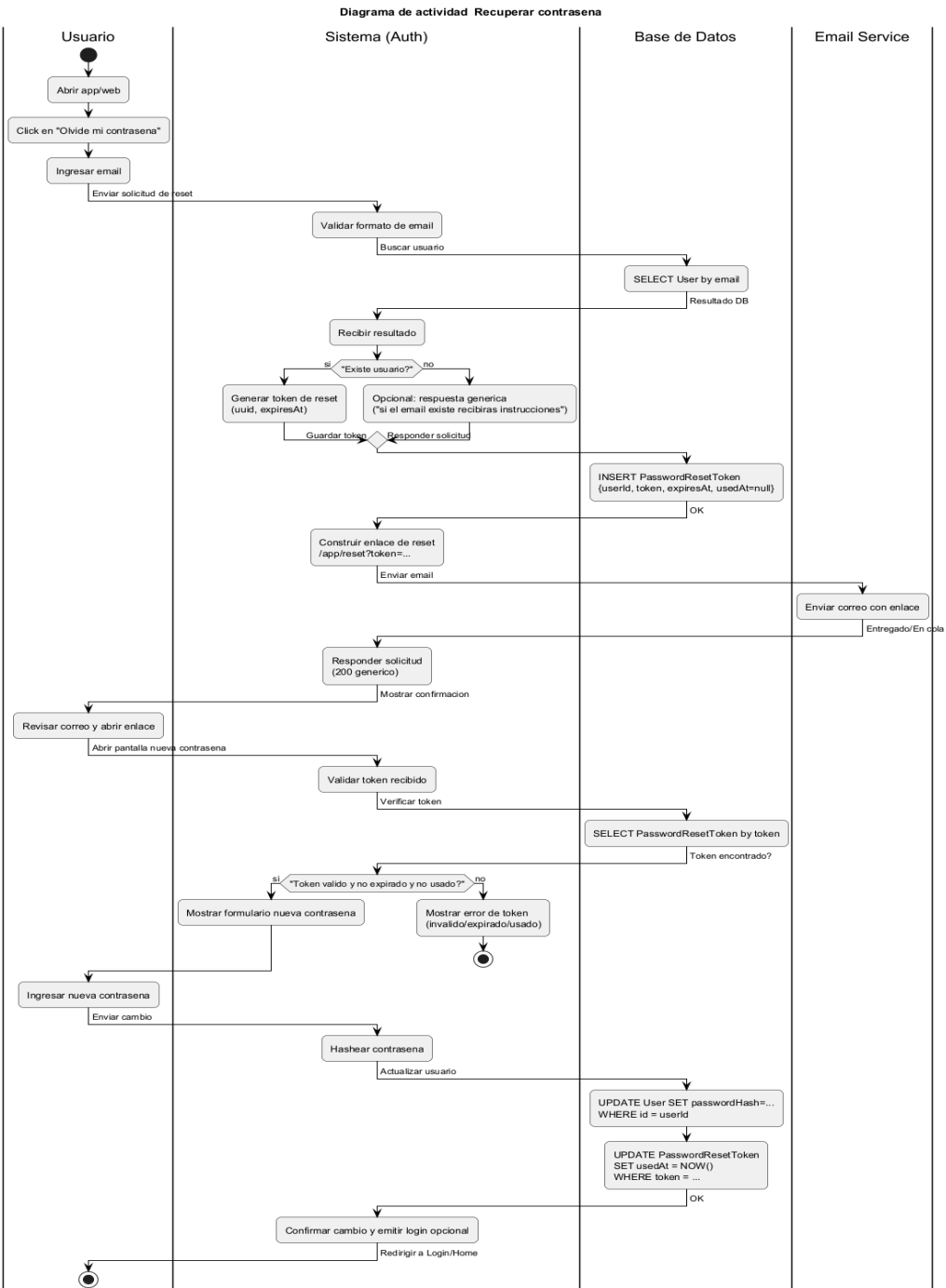
Para los diagramas de **Registrar Mascota** y **Generar Receta/Plan con IA**, se asume que el usuario ya ha completado el proceso de **Login** correctamente.

- **Login:** Representa el proceso de autenticación del usuario, ya sea mediante correo y contraseña o utilizando Google OAuth. Incluye las validaciones de credenciales y la generación de tokens de acceso (JWT).
- **Recuperar Contraseña:** Describe el flujo mediante el cual un usuario solicita la recuperación de su cuenta. Se ilustra el envío del enlace de restablecimiento a través del servicio de correo y la posterior actualización de la contraseña.
- **Registrar Mascota:** Muestra el proceso para crear un nuevo registro de mascota, validando duplicados, almacenando la información básica y permitiendo adjuntar una foto opcional.
- **Generar Receta/Plan con IA:** Detalla el flujo de generación automática de un plan nutricional personalizado. Incluye la obtención del perfil de la mascota, el uso opcional de ingredientes de la despensa, el envío de datos al motor de inteligencia artificial y la recepción de las recetas generadas.
- **Membresía Premium:** Este proceso muestra cómo el usuario gestiona la actualización, activación o cancelación de su plan dentro de la aplicación, incluyendo la validación del pago a través de la pasarela y la respuesta del sistema ante un resultado aprobado o rechazado. Cuando el pago es exitoso, la cuenta se actualiza y se muestran los beneficios del plan Premium, mientras que si el pago falla, el usuario es notificado y el plan se mantiene sin cambios. Además, se contempla la cancelación programada, manteniendo el acceso activo hasta el final del ciclo vigente.

Login



Recuperar contraseña



Registrar Mascota

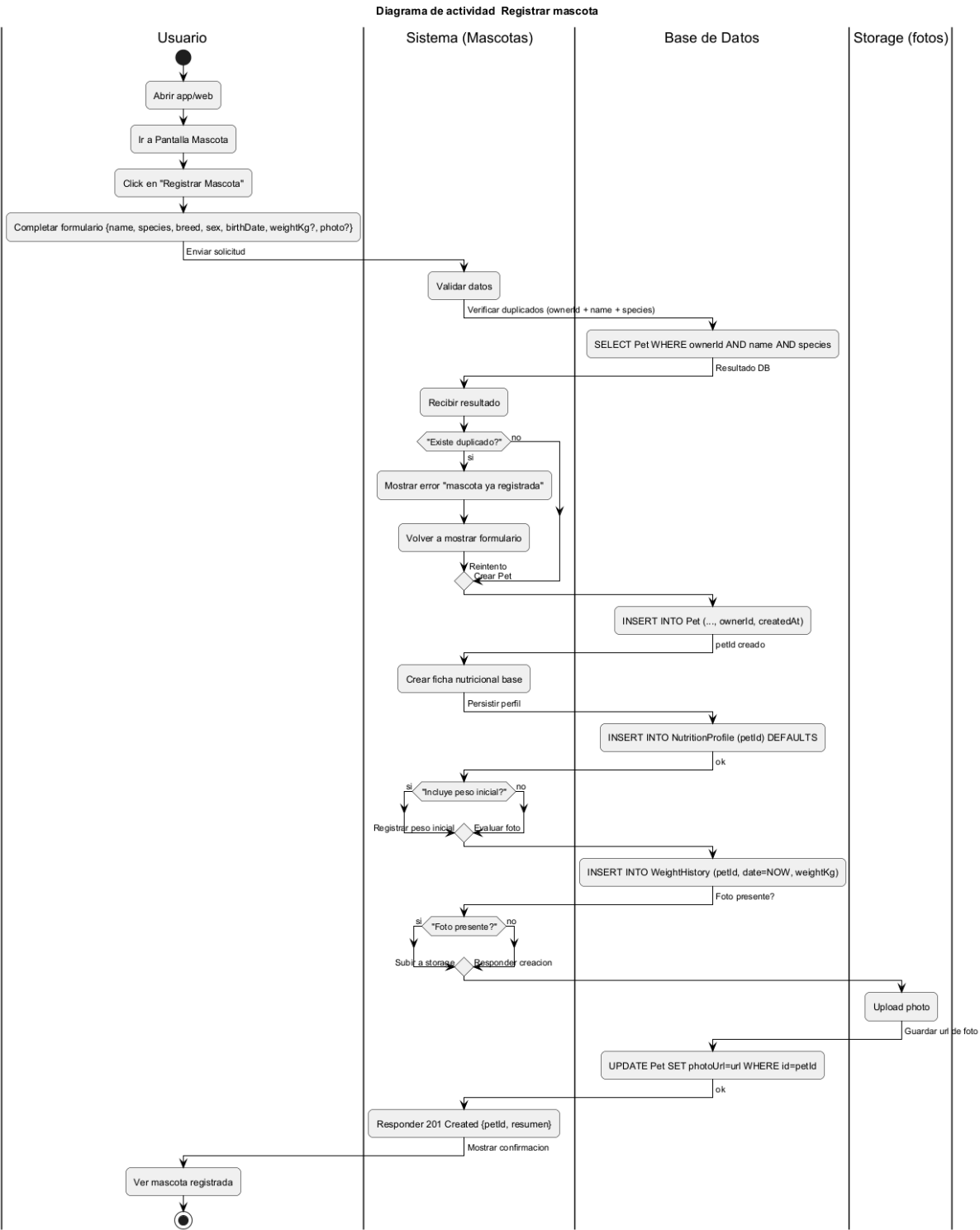


Diagrama de actividad Generar receta/plan con IA

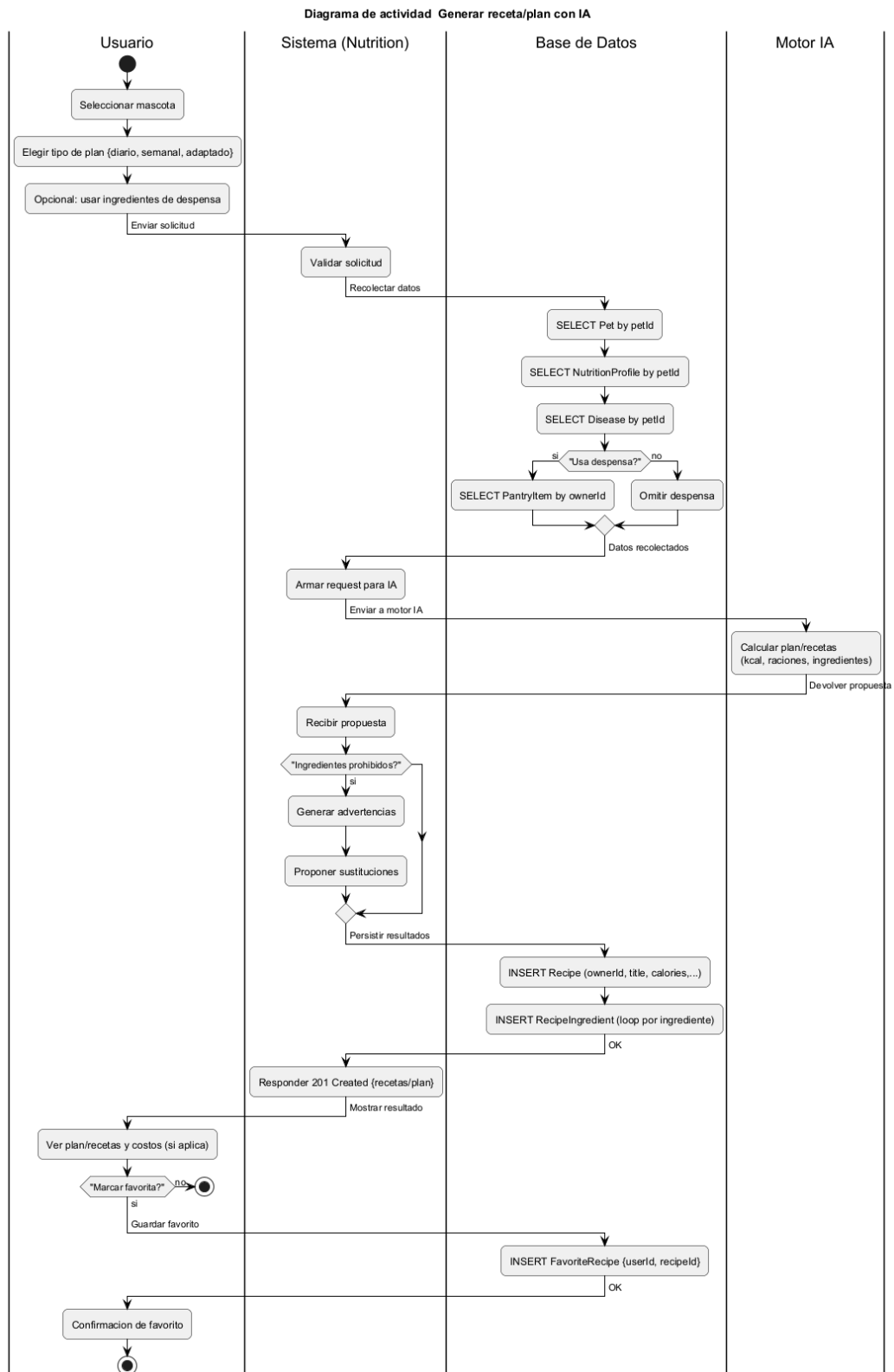
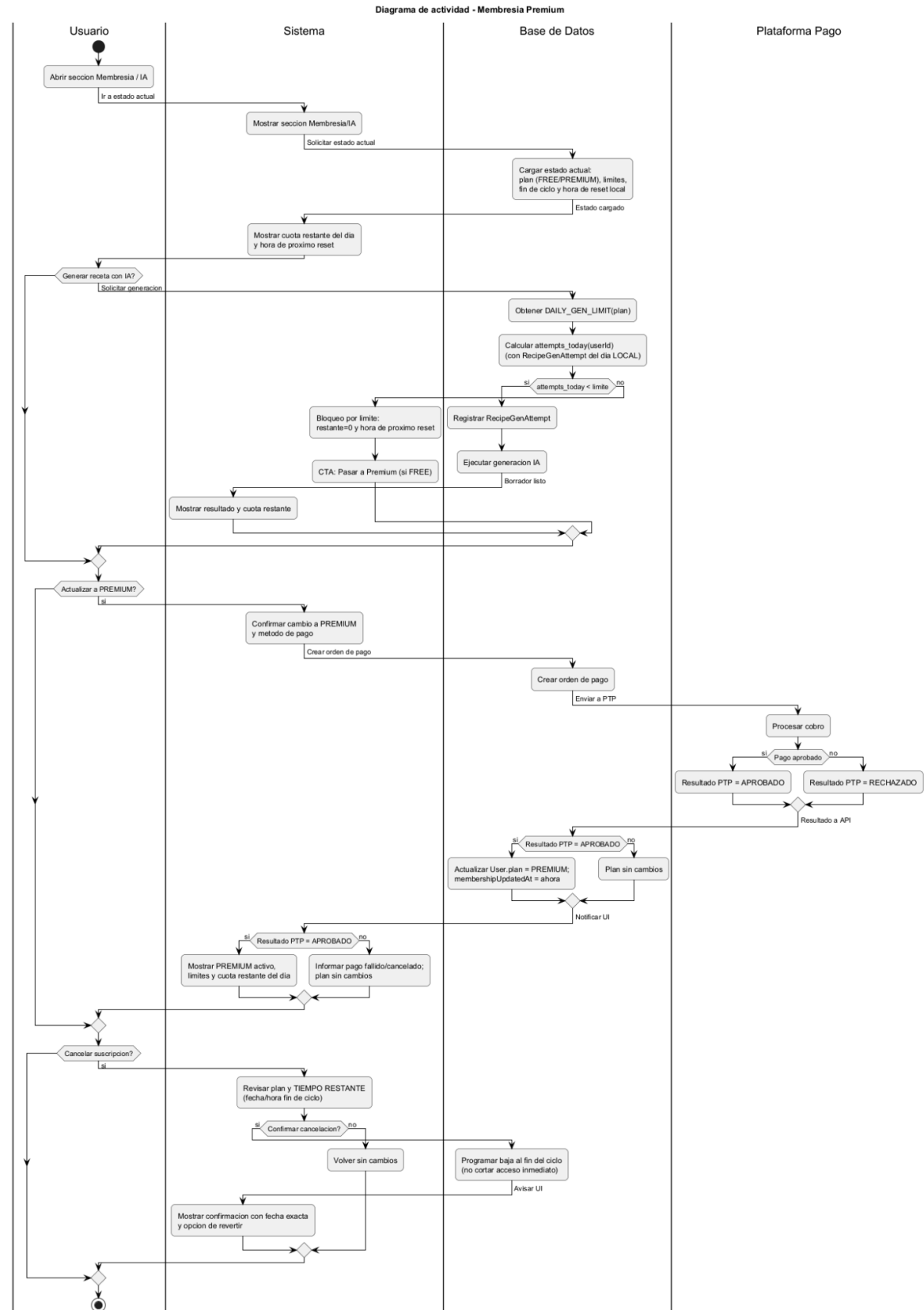
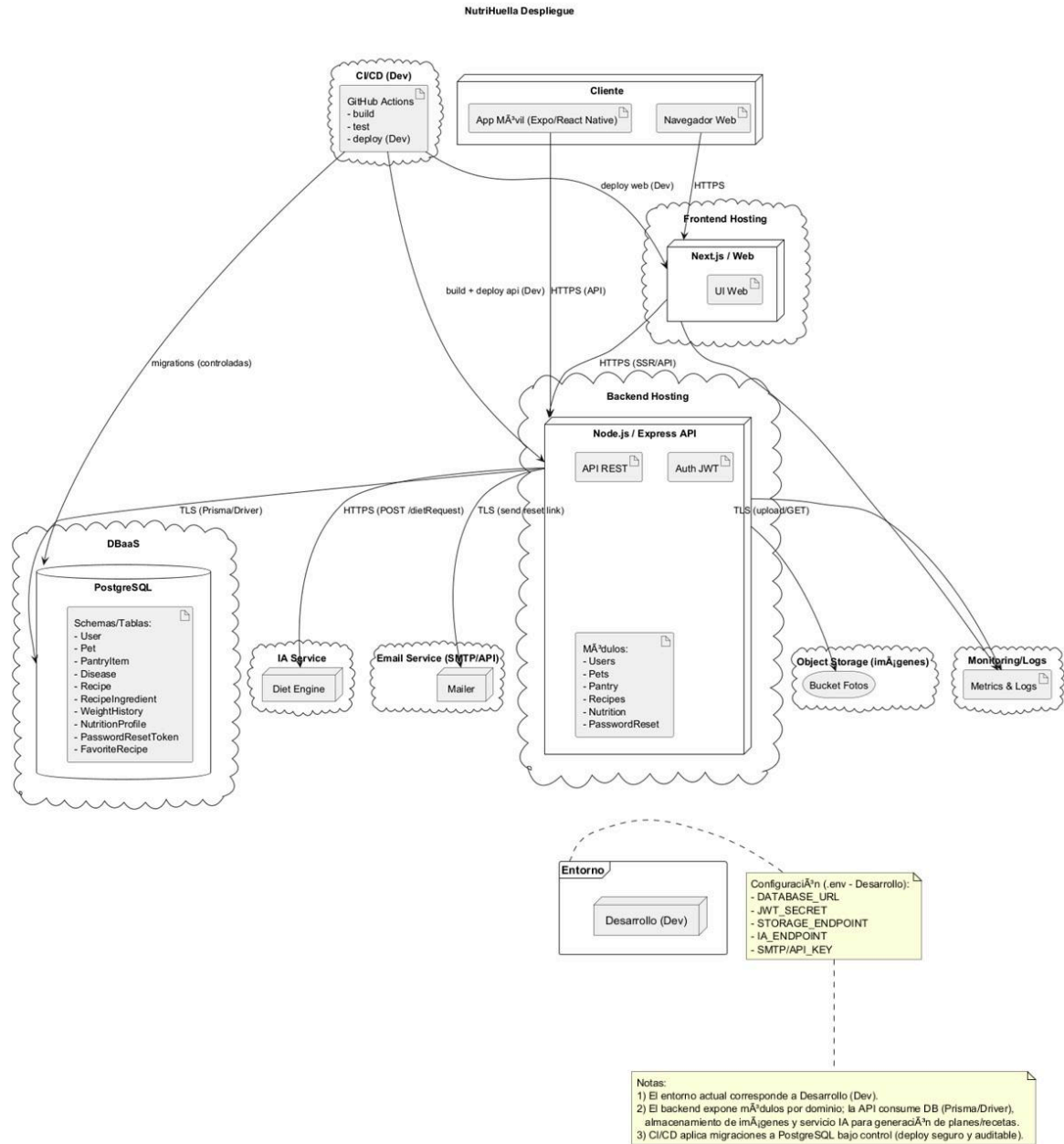


Diagrama de Actividad Membresía Premium



3.4 Vista de Fisica

Diagrama de despliegue



La presente vista describe la **arquitectura fÃ­sica y de despliegue del sistema NutriHuella** en su entorno actual de **Desarrollo (Dev)**. Este entorno integra todos los componentes necesarios para el funcionamiento, pruebas y validaciÃ³n del sistema antes de una eventual migraciÃ³n a producciÃ³n.

Descripción general

NutriHuella está compuesto por una arquitectura **cliente-servidor distribuida**, con separación clara entre las capas de presentación, lógica de negocio y persistencia de datos. Los componentes principales incluyen el **frontend web y móvil**, un **backend centralizado** con servicios REST, una **base de datos relacional PostgreSQL**, y servicios externos de apoyo como almacenamiento de imágenes, motor de inteligencia artificial y envío de correos electrónicos.

Componentes principales

1. Cliente

- Incluye el **Navegador Web** y la **App Móvil (Expo/React Native)**.
- Ambas interactúan con el backend a través de peticiones **HTTPS** seguras.

2. Frontend Hosting

- Implementado con **Next.js**, responsable de la interfaz de usuario (UI Web).
- Consume los servicios de la API mediante **JSON sobre HTTPS (SSR/API)**.

3. Backend Hosting

- Basado en **Node.js y Express**, encargado de procesar las solicitudes del cliente.
- Exposición de una **API REST** que agrupa módulos por dominio:
 - **Users, Pets, Pantry, Recipes, Nutrition, y PasswordReset.**
- Autenticación gestionada mediante **JWT** para garantizar sesiones seguras.

4. Base de Datos (DBaaS – PostgreSQL)

- Implementa las tablas definidas por el modelo de dominio: User, Pet, PantryItem, Recipe, WeightHistory, NutritionProfile, FavoriteRecipe, entre otras.
- El acceso se realiza mediante **TLS** y el ORM **Prisma**, que controla migraciones de esquema.

5. Servicios Externos

- **Object Storage:** almacena imágenes de mascotas y otros recursos multimedia.

- **IA Service (Diet Engine):** motor de inteligencia artificial que genera planes y recetas nutricionales personalizadas.
- **Email Service (SMTP/API):** gestiona el envío de correos electrónicos de verificación y recuperación de contraseñas.

6. CI/CD (Desarrollo)

- Gestionado por **GitHub Actions**, que ejecuta las etapas de *build*, *test* y *deploy*.
- Las migraciones a la base de datos se aplican de forma **controlada y segura**, garantizando integridad y trazabilidad.

7. Monitoring / Logs

- Centraliza métricas y registros del sistema para auditoría, rendimiento y diagnóstico de errores.

Entorno y configuración

El sistema opera en un único entorno denominado **Desarrollo (Dev)**.

Las configuraciones se manejan mediante un archivo **.env**, que contiene los siguientes parámetros clave:

- DATABASE_URL — conexión a la base de datos PostgreSQL.
- JWT_SECRET — clave de firma de tokens de autenticación.
- STORAGE_ENDPOINT — dirección del servicio de almacenamiento.
- IA_ENDPOINT — dirección del motor de IA.
- SMTP/API_KEY — clave de autenticación del servicio de correo.

Resumen del flujo de despliegue

1. El desarrollador realiza cambios en el código fuente y los publica en GitHub.
2. **GitHub Actions** ejecuta los procesos de construcción (*build*), prueba (*test*) y despliegue (*deploy*).
3. El **frontend (Next.js)** y el **backend (Express)** se despliegan automáticamente en sus entornos respectivos.
4. Las migraciones de base de datos se aplican de manera controlada hacia **PostgreSQL**.
5. Los clientes (web y móvil) acceden al sistema de forma segura vía **HTTPS**,

interactuando con la API y los servicios externos.
0

3.5 Vista de Escenarios/Caso de uso

Casos de uso

A continuación se presentan los modelos de casos de uso, para los diferentes roles:

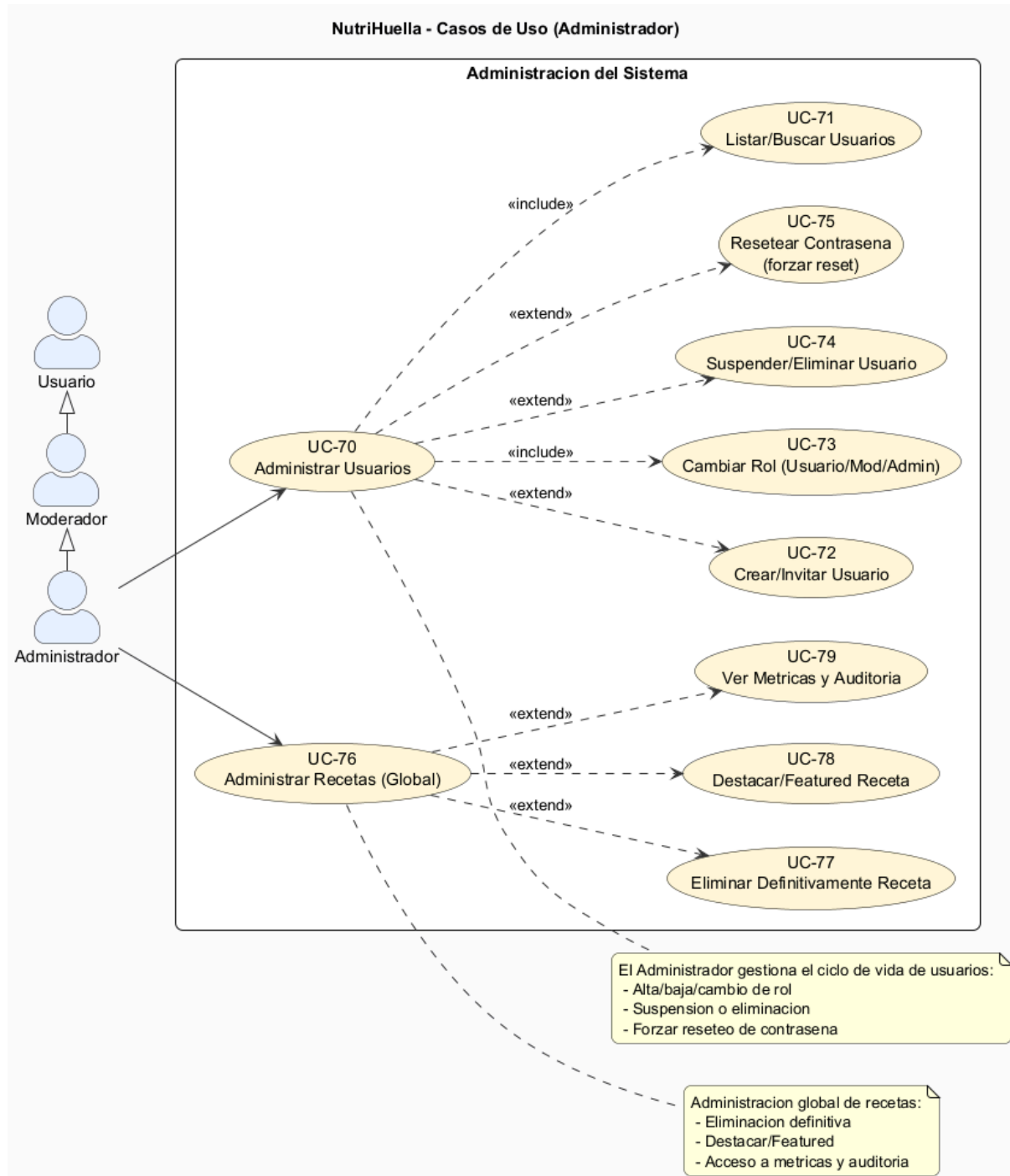


Ilustración 1. Caso de Uso Administrador.

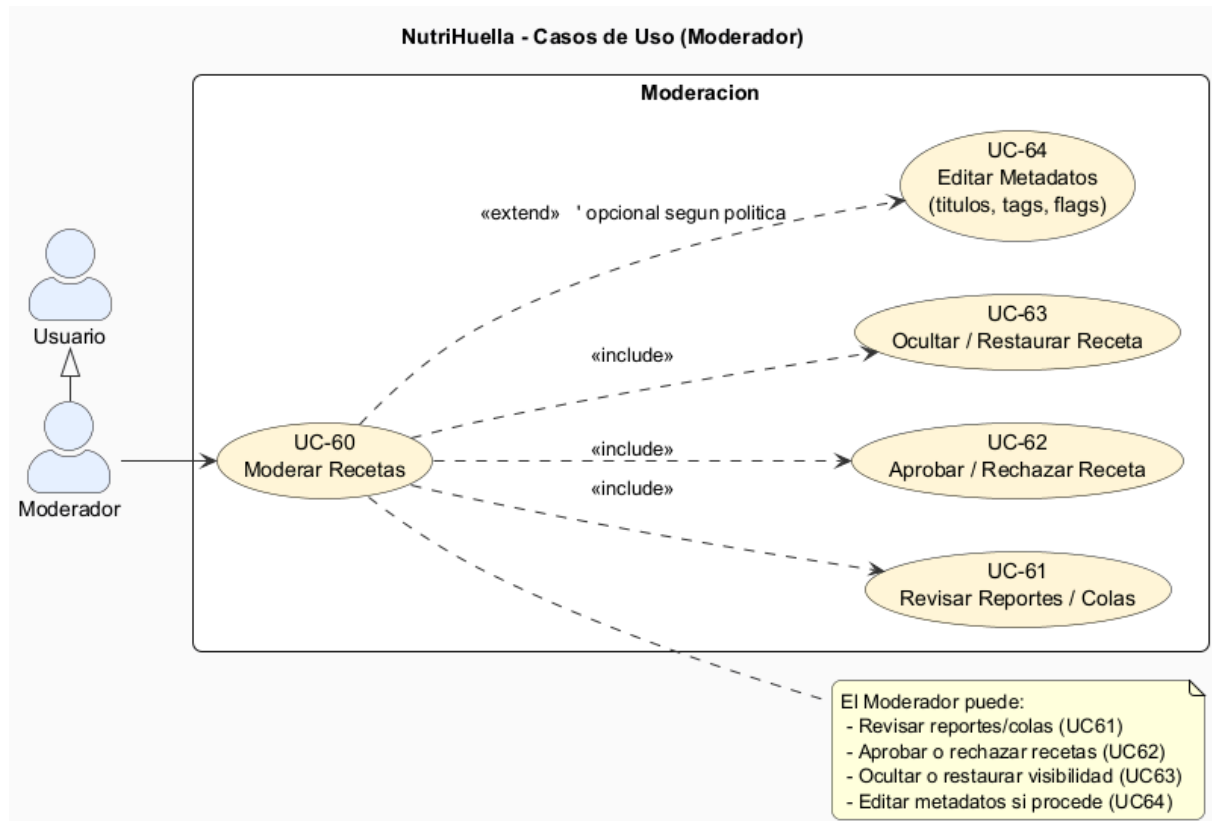


Ilustración 2. Caso de uso Moderador.

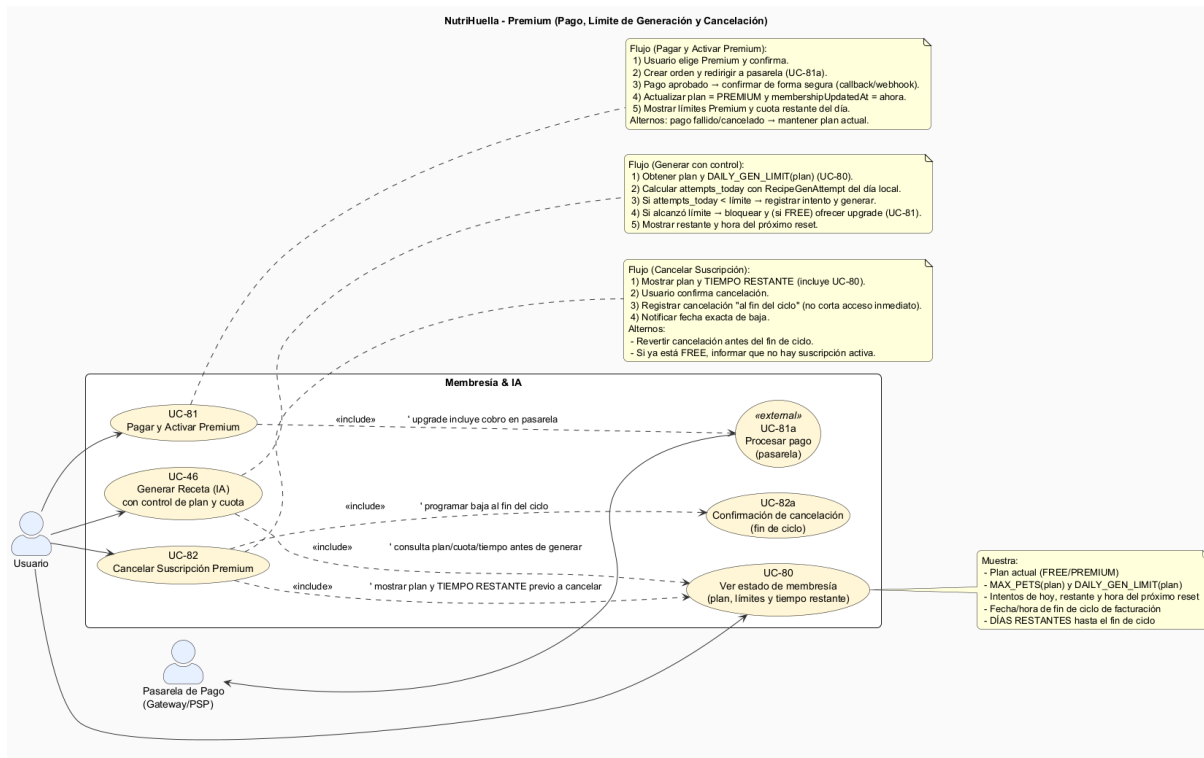


Ilustración 4. Caso de uso membresía Premium

3.6 Especificación de Casos de Uso Relevantes

La selección de los casos de uso más relevantes para **NutriHuella** se basó en: (i) criticidad para el funcionamiento del sistema, (ii) impacto en la experiencia del usuario, (iii) dependencia de otros procesos (pre/postcondiciones) y (iv) presencia de escenarios de calidad (seguridad, disponibilidad y usabilidad).

Los casos clave cubren **autenticación**, la **creación de perfil de mascota**, la **generación y gestión de recetas con IA**, la **despensa/ingredientes**, y las funciones de **moderación y administración**.

Código	Nombre	Actores	Prioridad
UC-12	Registrarse	Usuario, Sistema	Muy Alta
UC-11	Iniciar sesión	Usuario, Sistema, Proveedor de Identidad (OAuth)	Muy Alta
UC-13	Recuperar contraseña	Usuario, Sistema	Alta
UC-21	Registrar mascota	Usuario, Sistema	Muy Alta
UC-24	Completar perfil nutricional (de mascota)	Usuario, Sistema	Muy Alta
UC-26	Registrar peso histórico (de mascota)	Usuario, Sistema	Alta
UC-30	Gestionar despensa	Usuario, Sistema	Alta
UC-31	Registrar ingrediente	Usuario, Sistema	Alta
UC-33	Eliminar ingrediente(s)	Usuario, Sistema	Media
UC-40	Generar receta (IA)	Usuario, Sistema	Muy Alta
UC-41	Control de plan y cuota diaria	Usuario, Sistema	Media
UC-43	Guardar receta	Usuario, Sistema	Alta
UC-50	Mis recetas	Usuario, Sistema	Alta
UC-53	Marcar/Desmarcar favorita	Usuario, Sistema	Media
UC-60	Moderar recetas (paquete)	Moderador, Sistema	Alta
UC-62	Aprobar/Rechazar receta	Moderador, Sistema	Muy Alta
UC-63	Ocultar/Restaurar receta	Moderador, Sistema	Alta
UC-70	Administrar usuarios (paquete)	Administrador, Sistema	Alta
UC-73	Cambiar rol (Usuario/Mod/Admin)	Administrador, Sistema	Alta
UC-74	Suspender/Eliminar usuario	Administrador, Sistema	Alta
UC-79	Ver métricas y auditoría	Administrador, Sistema	Media
UC-81	Pagar y Activar premium	Usuario, Administrador y Moderador	Alta

Tabla 1. Caso de Uso Relevantes.

Escenarios (Funcionales)

Escenario-F1 — Autenticación (incluye OAuth)

Actor: Usuario

Precondición: Cuenta existente

Disparador: “Iniciar sesión”

Flujo principal:

1. El usuario selecciona “**Iniciar Sesión (UC-11)**”.
- 2.
3. El sistema muestra el formulario de autenticación.
4. El usuario ingresa correo y contraseña.
5. El sistema **valida las credenciales (UC-15 Validar Credenciales)**.
6. *(Extend)* Si el usuario elige ingresar con Google, se ejecuta “**Autenticar con Proveedor (OAuth) (UC-16)**”.
7. Una vez validados los datos, el sistema crea el token de sesión y muestra el “**Menú Principal (UC-17)**”.

Flujos alternativos / excepciones:

- **A1:** Credenciales incorrectas → se informa error y se permite reintentar.
- **A2:** El usuario olvidó su contraseña → puede acceder a “**Recuperar Contraseña (UC-13)**”.
- **E1:** Fallo en conexión OAuth → se notifica y se ofrece autenticación manual.

Resultado:

Sesión iniciada; acceso al menú principal.

UC asociados: UC-11, UC-15, UC-16, UC-17, (UC-13 opcional)

Escenario-F2 — Registro de usuario por email

Actor: Usuario

Precondición: No autenticado

Disparador: "Crear cuenta"

Flujo principal:

1. El usuario selecciona **"Registrarse / Crear Cuenta (UC-12)"**.
2. El sistema solicita datos mínimos: nombre, correo y contraseña.
3. El usuario ingresa los datos y confirma.
4. El sistema **valida la información (UC-15 Validar Credenciales)**.
5. Si es correcta, crea la cuenta y muestra mensaje de confirmación.
6. Se ejecuta **"Iniciar Sesión (UC-11)"** automáticamente o se redirige al formulario.
7. Se despliega el **"Menú Principal (UC-17)"** al completar la autenticación.

Flujos alternativos / excepciones:

- **A1:** Correo ya registrado → se sugiere usar **"Recuperar Contraseña (UC-13)"**.
- **E1:** Error en validación → se informa y se solicita reintentar.

Resultado:

Cuenta creada y sesión activa.

UC asociados: UC-12, UC-15, UC-11, UC-17, (UC-13 opcional)

Escenario-F3— Alta de mascota y perfil nutricional

Actor: Usuario

Precondición: Sesión iniciada

Disparador: "Registrar mascota"

Flujo principal:

1. El usuario selecciona **"Registrar Mascota (UC-21)"**.
2. Ingresa datos generales: nombre, edad, peso, especie.
3. El sistema solicita completar **"Perfil Nutricional (UC-24)"** con datos de dieta y objetivos.
4. *(Extend)* Puede registrar **"Enfermedades (UC-25)"** y **"Peso Histórico (UC-26)"**.

5. El sistema guarda la información y actualiza la lista de mascotas.

Flujos alternativos / excepciones:

- **A1:** Si el usuario omite datos obligatorios, se solicita completarlos.
- **E1:** Error al guardar datos → se informa y se mantiene el formulario abierto.

Resultado:

Mascota registrada con perfil nutricional asociado.

UC asociados: UC-21, UC-24, (UC-25, UC-26)

Escenario-F4 — Gestionar despensa (alta, edición, eliminación)

Actor: Usuario

Precondición: Sesión iniciada

Disparador: “Despensa”

Flujo principal:

1. El usuario accede a “**Gestionar Despensa (UC-30)**”.
2. El sistema **lista ingredientes disponibles (UC-32)**.
3. El usuario puede “**Registrar Ingrediente (UC-31)**”, agregando nombre y cantidad.
4. También puede “**Eliminar Ingrediente(s) (UC-33)**” seleccionando uno o varios.
5. El sistema actualiza la despensa y confirma los cambios.

Flujos alternativos / excepciones:

- **A1:** Si se intenta agregar un ingrediente duplicado, se muestra advertencia.
- **E1:** Error de conexión o base de datos → se guarda temporalmente local y reintenta.

Resultado:

Despensa actualizada correctamente.

UC asociados: UC-30, UC-31, UC-32, UC-33

Escenario-F5 — Generar receta con IA usando la despensa

Actor: Usuario

Precondición: ≥ 1 mascota registrada (UC-21) y perfil nutricional (UC-24)

Disparador: "Generar receta"

Flujo principal:

1. El usuario selecciona "**Generar Receta (UC-40)**".
2. El sistema solicita "**Seleccionar Mascota (UC-41)**".
3. *(Extend)* El usuario puede "**Usar Ingredientes de Despensa (UC-42)**".
4. El sistema ejecuta el motor de IA y genera una propuesta nutricional.
5. El usuario "**Guardar Receta (UC-43)**".
6. *(Extend)* Puede "**Marcar como Favorita (UC-44)**" al guardar.
7. Se muestra la receta generada y guardada exitosamente.

Flujos alternativos / excepciones:

- **A1:** Sin conexión al motor IA → se muestra mensaje y se permite reintentar.
- **E1:** Error al guardar → se informa y no se marca como favorita.

Resultado:

Receta generada, guardada y (opcionalmente) marcada favorita.

UC asociados: UC-40, UC-41, UC-42, UC-43, UC-44

Escenario-F6 — Consultar y gestionar "Mis Recetas"

Actor: Usuario

Precondición: Sesión iniciada; existen recetas previas

Disparador: "Mis Recetas"

Flujo principal:

1. El usuario accede a "**Mis Recetas (UC-50)**".
2. El sistema **muestra las recetas guardadas (UC-51)**.
3. *(Extend)* Puede "**Ver Historial de Recetas Generadas (UC-52)**".
4. El usuario puede "**Marcar/Desmarcar Favorita (UC-53)**".

5. También puede **“Eliminar Receta (UC-54)”** si lo desea.
6. El sistema actualiza la vista con las modificaciones.

Flujos alternativos / excepciones:

- **A1:** Sin recetas → se muestra mensaje y opción para generar una nueva (**UC-40**).
- **E1:** Error de carga → se solicita reintentar más tarde.

Resultado:

Historial de recetas gestionado correctamente.

UC asociados: UC-50, UC-51, UC-52, UC-53, UC-54

Escenario-F7 — Moderar receta reportada

Actor: Moderador

Precondición: Contenido pendiente en cola

Disparador: “Cola de moderación”

Flujo principal:

1. El moderador ingresa a **“Moderar Recetas (UC-60)”**.
2. Incluye **“Revisar Reportes / Colas (UC-61)”** para ver pendientes.
3. Incluye **“Aprobar / Rechazar Receta (UC-62)”** según evaluación.
4. Incluye **“Ocultar / Restaurar Receta (UC-63)”** si aplica.
5. (*Extend*) **“Editar Metadatos (UC-64)”** cuando se requiere ajustar título o etiquetas.

Flujos alternativos / excepciones:

- **A1:** Receta duplicada o ya resuelta → se omite acción.
- **E1:** Error al guardar decisión → reintento automático.

Resultado:

Receta moderada; acción registrada en auditoría.

UC asociados: UC-60, UC-61, UC-62, UC-63, UC-64

Escenario-F8 — Administración de usuarios y roles

Actor: Administrador

Precondición: Políticas definidas

Disparador: “Administración de usuarios”

Flujo principal:

1. El administrador abre “**Administrar Usuarios (UC-70)**”.
2. Incluye “**Listar / Buscar Usuarios (UC-71)**”.
3. (*Extend*) Según la acción:
 - “**Crear / Invitar Usuario (UC-72)**”
 - “**Suspender / Eliminar Usuario (UC-74)**”
 - “**Resetear Contraseña (UC-75)**”
4. Incluye “**Cambiar Rol (UC-73)**” para modificar privilegios.
5. (*Extend*) “**Ver Métricas y Auditoría (UC-79)**” para registrar cambios.

Flujos alternativos / excepciones:

- **A1:** El usuario intenta modificarse a sí mismo → acción bloqueada.
- **E1:** Error de conexión → se pospone la acción y se notifica.

Resultado:

Usuarios y roles administrados correctamente; cambios registrados.

UC asociados: UC-70, UC-71, UC-72, UC-73, UC-74, UC-75, UC-79

Escenario-F9 — Generar receta (IA) con control de plan y cuota diaria

Actores: Usuario (primario), Sistema

Prioridad: Muy Alta

Breve: El usuario solicita una receta generada por IA. Antes de ejecutar, el sistema valida plan (FREE/PREMIUM) y cuota diaria de intentos; registra el intento y, si procede, entrega la receta.

Precondiciones

- Usuario autenticado y con cuenta activa.
- Existe al menos una mascota si la generación es específica de mascota (opcional).
- Configurados los límites por plan (MAX_PETS, DAILY_GEN_LIMIT).

Postcondiciones

- Siempre que se dispara la acción, queda **registrado** un RecipeGenAttempt (si pasa el control de cuota).
- Si el usuario **acepta/guarda** el resultado, queda creada una Recipe asociada al usuario (y opcionalmente a una mascota).
- Si se rechaza por límite/plan, no se registra intento ni receta, y se informa el motivo con CTA.

Flujo principal

1. **Solicitar generación:** el usuario envía parámetros (mascota opcional, objetivo, restricciones, etc.).
2. **Resolver plan y límites:** el sistema lee User.plan y límites vigentes (DAILY_GEN_LIMIT(plan)).
3. **Calcular intentos de hoy:** el sistema computa attempts_today(userId) contando RecipeGenAttempt del día **local del usuario**.
4. **Evaluar cuota:**
 - Si attempts_today < DAILY_GEN_LIMIT(plan), **continuar**; de lo contrario ir a *Flujo alternativo A*.
5. **Registrar intento:** crear RecipeGenAttempt con userId, petId (si aplica) y createdAt.
6. **Generar receta:** ejecutar IA y retornar el borrador (con title, content JSON válido, difficultyId opcional).

7. **Guardar opcional:** si el usuario confirma “Guardar receta”, crear Recipe (ver UC-43) y mostrar en “Mis recetas”.
8. **Mostrar cuota restante:** calcular $\text{remaining} = \text{DAILY_GEN_LIMIT} - \text{attempts_today_actualizado}$ y la hora del **próximo reset local**.

Flujos alternos / Excepciones

A. Límite diario alcanzado

A1. El sistema informa: “Has alcanzado tu límite diario **{DAILY_GEN_LIMIT(plan)}**.”

Restante: **0**. Intenta nuevamente a las **{horaResetLocal}**.”

A2. Si el plan es FREE, ofrecer CTA “Mejorar a Premium” (navega a actualización de plan).

A3. Termina sin registrar intento ni receta.

B. Requiere Premium (feature marcada como premium)

B1. Si la generación solicitada corresponde a una función premium, validar `User.plan = PREMIUM`.

B2. Si es FREE, informar el requisito y ofrecer CTA de upgrade.

B3. Termina sin registrar intento.

C. Datos insuficientes

C1. Faltan parámetros mínimos (p. ej., sin mascota cuando el modo lo exige).

C2. Informar campos requeridos y permitir corregir.

D. Error de IA o time-out

D1. Registrar el intento (ya hecho en paso 5).

D2. Informar “No se pudo generar” y sugerir reintentar; no se crea Recipe.

Reglas de negocio aplicables

- BN-51, BN-52, BN-53, BN-54, BN-55, BN-56, BN-57, BN-58, BN-59.
- (y, por consistencia general: BN-01, BN-03, BN-04, BN-39, BN-41, BN-49).

Escenarios Calidad (No Funcionales)

Escenario-Q1 — Seguridad de acceso a recetas privadas

Atributo de Calidad Asociado: Seguridad

Subcaracterística: Control de acceso / Autorización

Descripción:

Asegurar que solo el propietario de la receta (o roles autorizados: Moderador/Administrador) pueda acceder, modificar o eliminar recetas privadas.

Fuente del Estímulo: Usuario no autorizado

Estímulo: Intento de acceder a la receta de otro usuario (por URL/ID directa).

Artefacto: API de Recetas; Middleware de Autenticación/Autorización (JWT + políticas de rol/propietario).

Ambiente: Operación normal; sesión iniciada o token provisto.

Respuesta: Validar JWT y propiedad/rol; bloquear acceso si no corresponde y registrar auditoría.

Medida de Respuesta: Respuesta **403 Forbidden** y evento de auditoría registrado en < 1 s.

Flujo del Escenario:

1. Cliente solicita una receta por ID.
2. Middleware valida **JWT** (firma/expiración).
3. Se verifica **propietario** de la receta o **rol** autorizado.
4. Si la verificación falla → **403**; se registra auditoría.
5. Si pasa → se retorna la receta.

Escenario-Q2 — Rendimiento del motor de recetas

Atributo de Calidad Asociado: Eficiencia de desempeño

Sub Característica: Rendimiento (tiempo de respuesta)

Descripción:

Garantizar tiempos de respuesta adecuados del motor de IA al generar recetas, sin degradar UX.

Fuente del Estímulo: Usuario concurrente / carga de pruebas

Estímulo: Solicitudes de “Generar receta” en paralelo.

Artefacto: Nutrition Service (IA), API Gateway/Backend, cache.

Ambiente: Hora pico, despensa con ≤ 20 ítems.

Respuesta: Procesar y responder en tiempo objetivo; registrar latencias.

Medida de Respuesta: **p95 < 3 s**; métricas de latencia registradas; fallback de mensaje si excede SLO.

Flujo del Escenario:

1. N usuarios autenticados invocan **Generar Receta**.
2. API enruta a **Nutrition Service** (usa despensa/perfil).
3. Se calcula la receta; se **mide latencia** por request.
4. Si $p95 \leq 3 \text{ s}$ → éxito.
5. Si excede → **mensaje de fallback** y registro para tuning.

Escenario Q3 — Disponibilidad ante falla parcial (Nutrition Service)

Atributo de Calidad Asociado: Fiabilidad

Sub Característica: Disponibilidad / Tolerancia a fallos

Descripción:

Ante la caída del servicio de IA, el sistema mantiene operativas el resto de funciones y comunica el estado degradado.

Fuente del Estímulo: Falla de componente externo (IA)

Estímulo: Timeout o error del Nutrition Service durante generación.

Artefacto: Nutrition Service; UI de Recetas; backend con manejador de fallos/reintentos.

Ambiente: Falla parcial; demás servicios operativos.

Respuesta: Mostrar estado degradado, registrar evento y permitir reintento diferido.

Medida de Respuesta: **0 errores fatales**; mensaje claro en UI; reintento programado en < 5 s.

Flujo del Escenario:

1. Usuario solicita **Generar Receta**.
2. Backend detecta **falla/timeout** del servicio IA.
3. UI muestra **estado degradado** y opciones (reintentar/luego).
4. Sistema **registra** el incidente y **agenda reintento**.
5. Al restablecerse IA, se procesa o se notifica al usuario.

Esc-Q4 — Integridad transaccional al guardar receta

Atributo de Calidad Asociado: Fiabilidad / Adecuación funcional

Sub Característica: Integridad / Consistencia (atomicidad)

Descripción:

Asegurar que **Guardar Receta (UC-43)** y **Marcar Favorita (UC-44)** ocurran de forma atómica (todo o nada).

Fuente del Estímulo: Usuario autenticado

Estímulo: Guardar receta y marcarla favorita en la misma acción.

Artefacto: Repositorios (DB), capa de servicio/transactions.

Ambiente: Operación normal con base de datos disponible.

Respuesta: Ejecutar ambas operaciones en una transacción.

Medida de Respuesta: Commit si ambas confirman; **rollback** si una falla; **sin estados intermedios**.

Flujo del Escenario:

1. Usuario confirma **Guardar** y “Marcar favorita”.
2. Servicio abre **transacción** (DB).
3. Inserta receta; asigna flag de favorito.
4. Si ambas OK → **commit**; si alguna falla → **rollback**.
5. UI muestra confirmación o error consistente.

Escenario-Q5 — Escalabilidad

Atributo de Calidad Asociado: Eficiencia de desempeño

Sub Característica: Capacidad / Escalabilidad

Descripción:

Soportar alta concurrencia de usuarios generando recetas en un intervalo corto sin timeouts masivos.

Fuente del Estímulo: Carga concurrente (evento/campaña)

Estímulo: 500 usuarios autenticados generan recetas en 10 minutos.

Artefacto: API Gateway, autoscaling, colas, Nutrition Service.

Ambiente: Pico de uso; distribución horizontal activa.

Respuesta: Mantener throughput estable y controlar colas.

Medida de Respuesta: Cola controlada $< N$; sin timeouts masivos; tasa de éxito sostenida.

Flujo del Escenario:

1. Llega pico de solicitudes a la API.
2. Balanceador distribuye; **autoscaling** agrega réplicas.
3. Se **aplican colas**/back-pressure si es necesario.
4. Se monitorea throughput y error rate.
5. Sin **timeouts masivos**; cola bajo umbral N.

Escenario-Q6 — Usabilidad

Atributo de Calidad Asociado: Usabilidad

Sub Característica: Accesibilidad y facilidad de uso

Descripción:

Verificar que el flujo “**Generar y Guardar receta**” sea fácil de usar en una **pantalla típica** (texto legible, buen contraste y botones claros).

Fuente del Estímulo: Usuario

Estímulo: Generar y guardar una receta.

Artefacto: Interfaz de NutriHuella.

Ambiente: Pantalla típica.

Respuesta: El usuario completa el flujo sin dificultad.

Medida de Respuesta: Se realiza en ≤ 5 pasos y ≥ 90 % de los usuarios lo logra sin ayuda.

Flujo del Escenario:

1. El usuario abre la app y selecciona **Generar Receta**.
2. Elige la mascota y confirma ingredientes.
3. **Guarda** la receta en **menos de 5 pasos**.
4. Los textos y botones se ven **claros y legibles**.

4 Patron

Cliente–Servidor

El proyecto **NutriHuella** implementa una **arquitectura basada en el patrón Cliente–Servidor**, complementada con el estilo **API RESTful**.

Este enfoque permite separar de manera clara las responsabilidades entre los distintos componentes del sistema, manteniendo la escalabilidad, la mantenibilidad y la interoperabilidad entre los módulos.

En esta arquitectura, el **cliente** está compuesto por dos interfaces de presentación:

- una **aplicación web** desarrollada en **Next.js**, y
- una **aplicación móvil** creada con **React Native (Expo)**.

Ambas interactúan con el **servidor central** a través de solicitudes **HTTP/HTTPS** en formato **JSON**, consumiendo los servicios expuestos por la **API REST**.

Esta comunicación permite que los clientes permanezcan ligeros, delegando toda la lógica de negocio, validaciones y persistencia de datos en el servidor.

El **servidor** está desarrollado con **Node.js y Express**, siguiendo una estructura modular organizada por dominios (Autenticación, Mascotas, Despensa, Recetas, Nutrición y Recuperación de Contraseña).

Cada módulo agrupa su propio conjunto de controladores, servicios y repositorios, lo que convierte al backend en **modular**, donde los módulos pueden evolucionar de forma independiente sin perder cohesión interna.

El servidor se encarga de autenticar a los usuarios mediante **JWT**, procesar las peticiones, ejecutar las reglas de negocio y comunicarse con los servicios externos.

Dentro de la capa de datos, se utiliza **PostgreSQL** como base de datos relacional principal, administrada mediante el ORM **Prisma**, lo que proporciona una abstracción segura y eficiente para las operaciones CRUD.

El sistema también se integra con otros servicios especializados, como el **motor de inteligencia artificial (Diet Engine)** para generar planes alimenticios personalizados, el **servicio de almacenamiento de imágenes (Object Storage)** para guardar fotografías de mascotas y el **servicio de correo (SMTP/API)** para el envío de enlaces de recuperación de contraseña.

El despliegue y control de versiones se gestionan mediante **GitHub Actions (CI/CD)**, que automatiza las tareas de compilación, prueba y despliegue tanto del frontend como del backend en el entorno de **Desarrollo (Dev)**, garantizando un ciclo de entrega continuo y auditable.

Gracias a esta organización, NutriHuella logra una arquitectura **escalable, mantenible y extensible**, en la que los cambios en el cliente o en el servidor no afectan el funcionamiento general del sistema.

El patrón Cliente–Servidor, reforzado por la exposición de una API REST, proporciona una base sólida para evolucionar el proyecto hacia entornos productivos o incluso hacia una futura arquitectura de microservicios, sin perder compatibilidad ni simplicidad en el mantenimiento.

5 Conclusion

El proyecto NutriHuella, enfocado en un interés real de las personas, demostró la consolidación de los aprendizajes de la carrera y construir un sistema completo: autenticación de usuarios, CRUD de mascotas y despensa, y una alimentación de datos a la IA más rápida, eficiente y veraz. La arquitectura seleccionada resultó adecuada para un MVP modular, escalable y mantenible; el uso de Prisma agilizó la evolución del modelo de datos sin perder coherencia entre capas. Sobre esa base incorporamos validaciones y servicios clave —incluida la pasarela de pago— y sostuvimos las decisiones con documentación y diagramas (incluido el modelo 4+1) que guiaron la implementación.

Un elemento diferenciador fue que, por decisión del equipo, no nos limitamos a los lenguajes y herramientas enseñados formalmente en la institución: optamos por un stack moderno elegido por cuenta propia (p. ej., frameworks web, ORM y capas de IA locales), y su adopción fue posible gracias a las competencias adquiridas durante la formación y al estudio autónomo y disciplinado. Pese a la difícil coordinación de agendas, se logró una colaboración efectiva en la que cada integrante aportó en múltiples frentes, evidenciando madurez técnica y de trabajo en equipo.

En síntesis, NutriHuella reafirma la importancia de una arquitectura clara, procesos ágiles y documentación viva; y pone en evidencia las distintas competencias adquiridas a lo largo de la carrera, pues se gestionó un proyecto desde la idea que nació en el mismo equipo, dando como resultado un producto escalable, que responde a los requerimientos de acuerdo al diseño definido.